Project Report On

Real-time Contextual Profanity Detection and Censorship Using Transformer-Based NLP

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science and Engineering

By

Nichol Joseph Sunil (U2203158)
Roshan Erattupuzha Joseph (U2203183)
Sagar S Korattiyil (U2203185)
Sahil Anas (U2203186)

Under the guidance of

Dr.Jincy J Fernandez

Department of Computer Science and Engineering
Rajagiri School of Engineering & Technology (Autonomous)
(Parent University: APJ Abdul Kalam Technological University)
Rajagiri Valley, Kakkanad, Kochi, 682039
March 2026

CERTIFICATE

This is to certify that the project report entitled ***"Real-time Contextual Profanity Detection and Censorship Using Transformer-Based NLP "*** *is a bonafide record of the work done by* ***Nichol Joseph Sunil (U2203158), Roshan Erattupuzha Joseph (U2203183), Sagar S Korattiyil (U2203185), Sahil Anas (U2203186)*** *submitted to the Rajagiri School of Engineering & Technology (RSET) (Autonomous) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology (B. Tech.) in "Computer Science and Engineering" during the academic year 2025-2026.*

Project Guide
Dr. Jincy J Fernandez
Associate Professor
Dept. of CSE
RSET

Project Co-ordinator
Ms. Jisha Mary Jose
Assistant Professor
Dept. of CSE
RSET

Head Of Department
Dr. Preetha K G
Professor
Dept. of CSE
RSET

ACKNOWLEDGMENT

We wish to express our sincere gratitude towards **Rev. Dr. Jaison Paul Mulerikkal CMI**, Principal of RSET, and **Dr. Preetha K.G.**, Head of the Department of Computer Science and Engineering, for providing us with the opportunity to undertake our project, *"Real-time Contextual Profanity Detection and Censorship Using Transformer-Based NLP"*.

We are highly indebted to our project coordinator, **Ms. Jisha Mary Jose**, Assistant Professor, Department of Computer Science and Engineering, for her valuable support.

It is indeed our pleasure and a moment of satisfaction for us to express our sincere gratitude to our project guide, **Dr. Jincy J Fernandez**, Associate Professor, Department of Computer Science and Engineering, for her patience and for all the priceless advice and wisdom she has shared with us.

Last but not the least, we would like to express our sincere gratitude towards all other teachers and friends for their continuous support and constructive ideas.

Nichol Joseph Sunil
Roshan Erattupuzha Joseph
Sagar S Korattiyil
Sahil Anas

Abstract

An advanced AI-powered framework is presented for real-time profanity detection and censorship in live audio-video streams. Traditional content moderation systems often rely on offline analysis or keyword-based filtering, limiting their effectiveness in handling context-aware and real-time scenarios. The proposed approach integrates speech recognition and natural language processing into a unified pipeline, where audio streams are transcribed using the Whisper model with precise word-level timestamps, and the resulting text is analyzed using a Transformer-based BERT classifier to detect offensive language, including contextually implied or disguised expressions.

Upon detection, real-time audio censorship is applied by inserting masking signals such as beep sounds while preserving synchronization with the video stream. The system architecture comprises modular components including Live Input Capture, Decoder, Audio Processing, Video Processing, and Output modules, ensuring efficient data flow and low-latency performance. All processing is performed locally to maintain data privacy and security, achieving a target F1-score of ≥ 0.85. Experimental evaluation demonstrates reliable transcription, accurate contextual detection, and seamless censorship, establishing the framework as a scalable solution for content moderation with potential extensions such as multilingual support and cloud-based deployment.

Contents

List of Abbreviations

AI	Artificial Intelligence
ASR	Automatic Speech Recognition
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
BiGRU	Bidirectional Gated Recurrent Unit
CNN	Convolutional Neural Network
CPU	Central Processing Unit
DL	Deep Learning
FFmpeg	Fast Forward Moving Picture Experts Group
GPU	Graphics Processing Unit
GRU	Gated Recurrent Unit
JSON	JavaScript Object Notation
LSTM	Long Short-Term Memory
MFCC	Mel-Frequency Cepstral Coefficients
MHA	Multi-Head Attention
NLP	Natural Language Processing
RNN	Recurrent Neural Network
STFT	Short-Time Fourier Transform
STT	Speech-to-Text
UDP	User Datagram Protocol
VAD	Voice Activity Detection
WER	Word Error Rate

List of Figures

List of Tables

Chapter 1

Introduction

This chapter introduces the concept of the project "Real-Time Contextual Profanity Detection and Censorship Using Transformer-Based NLP." It outlines the motivation behind developing an intelligent, automated system for moderating offensive language in live audio-video streams, the key challenges associated with real-time content moderation, and the technological approaches adopted to address them. The subsequent sections describe the problem context, system objectives, and the overall relevance of this work in the evolving landscape of AI-driven multimedia content moderation.

1.1 Background

The exponential growth of digital content across live streaming platforms, social media, and broadcasting networks has led to an increasing concern regarding the presence of offensive or abusive language in audio-video content. Managing and moderating such content in real time is a critical challenge for online platforms, as manual moderation is inefficient, inconsistent, and impractical at scale.

In recent years, the proliferation of live streaming and user-generated video content has revolutionized communication and entertainment. However, with this rise comes the challenge of moderating content that may contain hate speech, profanity, or offensive language. Traditional moderation techniques rely heavily on human intervention or post-processing approaches that are both time-consuming and ineffective for real-time scenarios.

Artificial Intelligence, particularly Transformer-based architectures like Whisper (for speech recognition) and BERT (for context-aware text understanding), has enabled the development of intelligent systems that can process human language with remarkable accuracy. Leveraging these technologies, this project aims to design a modular, real-time profanity detection and censorship system that operates automatically, ensuring clean and compliant live broadcasts across various digital media platforms.

1.2 Problem Definition

Live video broadcasts and online streaming platforms frequently encounter unintended use of offensive or inappropriate language, which can lead to viewer discomfort, reputational harm, and policy violations. Existing moderation systems are typically manual, delayed, or platform-dependent, making them unsuitable for instantaneous censorship during live sessions.

The core problem addressed by this project is the lack of an autonomous, real-time, and context-aware censorship mechanism capable of accurately detecting and censoring profane speech from live audio-video streams while maintaining audio-video synchronization and minimal latency.

1.3 Scope and Motivation

The scope of this project encompasses the design and implementation of a real-time, modular profanity detection and censorship system that operates on live or recorded video streams. The system performs end-to-end processing—from audio capture to censorship and reassembly—entirely in real time.

The design is structured into multiple functional modules as follows:

- **Live Input Capture Module:** Captures live audio-video input through the user's microphone and camera.
- **Decoder Module:** Separates audio and video streams for independent processing.
- **Audio Processing Module:** Performs transcription, profanity detection, and censorship using *Whisper* and *BERT* models.
- **Video Processing Module:** Maintains the original video quality and ensures synchronization with censored audio.
- **Output Module:** Recombines processed audio and video and displays the censored live feed.

The primary motivation for this project arises from the growing need to protect audiences from exposure to harmful or offensive content. Live streaming platforms, online classrooms, gaming broadcasts, and social media streams often face incidents where users use profane or inappropriate language, intentionally or otherwise.

Manual moderation is impractical for real-time situations, and existing automatic filters often lack contextual understanding, resulting in false censorship or missed detections. By leveraging the capabilities of AI and NLP, this project seeks to create a context-aware and efficient censorship framework that ensures safe, inclusive, and regulation-compliant live content delivery without degrading user experience.

1.4 Objectives

- To design and develop a real-time AI-based profanity detection and censorship system for live video streaming.
- To implement an Automatic Speech Recognition (ASR) module using Whisper to transcribe spoken audio with word-level timestamps.
- To design a Transformer-based NLP model (BERT) capable of detecting profane language with contextual understanding.
- To develop an automated audio censorship mechanism that inserts beep or mute sounds over detected offensive segments while maintaining audio-video synchronization.
- To ensure low-latency and high-accuracy performance suitable for live broadcasting (target F1-score ≥ 0.85).
- To process data locally, ensuring privacy and independence from external servers or cloud dependencies.
- To provide a modular and extensible system architecture capable of future expansion into multilingual and distributed environments.

1.5 Challenges

Even with advanced systems, several challenges hinder the reliable deployment of real-time profanity detection systems. Latency and performance constraints may introduce delays that affect synchronization. Transformer models may misinterpret slang, sarcasm, or contextual nuances, leading to false positives or false negatives.

Maintaining precise audio-video synchronization during censorship is technically complex. Background noise, overlapping speech, and poor audio quality can reduce ASR accuracy. Additionally, scaling the system to support multiple streams increases computational demand, and running advanced models locally requires significant hardware resources.

1.6 Assumptions

The following assumptions are made in the design and implementation of the proposed system:

- The input audio-video stream is captured through standard devices (microphone and webcam) with sufficient quality.
- The offensive word list and contextual dataset are pre-defined and continuously improvable through fine-tuning.
- The system operates locally with necessary hardware support (GPU/CPU capable of real-time inference).
- Network latency and streaming bandwidth are sufficient to support live processing without frame drops.
- Whisper and BERT models are properly optimized or fine-tuned for the chosen language.

1.7 Societal / Industrial Relevance

The proposed system has significant societal and industrial impact. From a societal perspective, it promotes safer digital communication by protecting audiences from exposure to abusive or harmful language. It supports parental control, educational content moderation, and ethical broadcasting practices.

From an industrial standpoint, the system can be adopted by OTT platforms, live streaming services, broadcasters, and educational institutions to ensure compliance with content policies and improve user experience. By automating moderation, it reduces human workload, ensures consistency, and enhances brand reputation through responsible media delivery.

1.8 Organization of the Report

Chapter 1 provides an overview of the project, including background, problem definition, scope, objectives, challenges, assumptions, and societal relevance. Chapter 2 presents a survey of related work on profanity detection, speech recognition, and content moderation systems. Chapter 3 describes the system design, architecture, and module decomposition. Chapter 4 provides a detailed technical description of the implementation. Chapter 5 presents and analyzes experimental results. Chapter 6 concludes the report and outlines future work directions.

1.9 Summary of Chapter

This chapter introduced the concept, motivation, and necessity behind developing a real-time contextual profanity detection and censorship system using Transformer-based NLP models. It outlined the project's objectives, problem statement, scope, assumptions, challenges, and societal importance. The chapter also described how the system aligns with modern AI-driven content moderation trends and serves as a foundation for future research in ethical, automated speech filtering. The next chapter, Literature Review, will explore existing systems, related works, and theoretical foundations that influenced the development of this project.

Chapter 2

Literature Survey

This chapter presents a comprehensive review of existing research related to real-time profanity detection, speech recognition, contextual natural language processing (NLP), and multimedia censorship systems. With the rapid growth of user-generated content on digital platforms, the need for intelligent systems capable of detecting and filtering offensive language in real-time has become increasingly important.

Recent advancements in deep learning, particularly transformer-based architectures and large-scale speech recognition models, have significantly improved the ability to process and understand both textual and audio data. This chapter explores these advancements by analyzing key research contributions in ASR, NLP, and multimodal systems, highlighting their strengths, methodologies, and limitations. A comparative analysis is also provided to identify research gaps and motivate the design of the proposed system.

2.1 A Fine-Tuned BERT-Based Transfer Learning Approach for Text Classification [1]

Traditional machine learning approaches such as TF-IDF combined with classifiers like Support Vector Machines (SVM) and Logistic Regression have been widely used for text classification. However, these methods fail to capture semantic meaning and contextual relationships between words, as they treat text as independent tokens rather than sequences with dependencies [1]. Earlier deep learning models such as CNNs and LSTMs attempted to improve performance but still struggle with long-range dependencies and bidirectional context understanding. Additionally, words often have multiple meanings depending on context, leading to ambiguity and incorrect classification. Training such models from scratch also requires large labeled datasets, which are expensive and time-consuming to obtain.

Objective:

- To utilize transfer learning with pre-trained BERT models for improved contextual understanding.

- To reduce dependency on manual feature engineering.
- To improve classification performance with limited labeled data.

Methodology

The methodology begins with preprocessing the text through cleaning, normalization, and tokenization using the WordPiece tokenizer. Special tokens such as [CLS] and [SEP] are added to represent sentence structure and classification tasks. The processed input is then fed into the BERT architecture, which consists of multiple transformer encoder layers with self-attention mechanisms that capture bidirectional contextual relationships. The pre-trained model is fine-tuned by adding a classification layer and training it on task-specific datasets using a low learning rate. This approach enables the model to adapt to specific tasks while preserving previously learned linguistic knowledge.

Key Findings

- Achieves superior contextual understanding compared to traditional models.
- Provides high accuracy and better generalization across domains [1].

Limitations

- High computational and memory requirements.
- Not suitable for low-resource real-time applications without optimization.

2.2 Robust Speech Recognition via Large-Scale Weak Supervision (Whisper) [2]

Traditional Automatic Speech Recognition (ASR) systems face significant challenges in real-world environments, including background noise, variations in speaker accents, and multilingual inputs [2]. These limitations reduce transcription accuracy and reliability, particularly in live streaming scenarios. Additionally, many ASR systems struggle to generate accurate word-level timestamps, which are essential for synchronization in real-time applications such as audio censorship and subtitle generation.

Objective

- To develop a robust ASR system capable of handling diverse audio conditions.
- To improve transcription accuracy and timestamp alignment.
- To support real-time and multilingual speech recognition.

Methodology

The methodology utilizes large-scale weakly supervised learning on multilingual datasets to improve generalization across various audio conditions. A transformer-based encoder-decoder architecture is employed for sequence modeling, where the encoder extracts features from speech signals and the decoder generates corresponding text. The system follows an end-to-end learning approach, directly mapping speech input to text output, thereby simplifying the pipeline and improving performance.

Key Findings

- High transcription accuracy across diverse accents and environments [2].
- Generates reliable word-level timestamps.
- Performs well in noisy real-world conditions.

Limitations

- High computational cost and memory usage.
- Latency issues in low-resource devices.

2.3 Design and Implementation of Fast Spoken Foul Language Recognition [3]

Detecting profanity directly from speech signals in real time is challenging due to variations in tone, pitch, pronunciation, and speaking style [3]. Traditional text-based methods require prior transcription, which introduces delays and reduces efficiency in live applications. Additionally, the lack of large and diverse datasets containing offensive speech limits model performance and generalization.

Objective

- To develop a lightweight model for real-time profanity detection.
- To reduce latency while maintaining acceptable accuracy.

Methodology

The methodology involves extracting acoustic features such as MFCC and Mel-spectrograms from speech signals, which capture important audio characteristics. A hybrid deep learning model is used, where CNN layers extract spatial features and LSTM layers model temporal dependencies in the speech sequence. Data augmentation techniques such as noise addition and

pitch shifting are applied to improve robustness. The model is trained on a custom-built dataset containing offensive speech samples.

Key Findings

- Achieves efficient real-time detection with reasonable accuracy.
- Lightweight architecture suitable for streaming applications.

Limitations

- Lacks contextual understanding of language.
- Performance depends heavily on dataset quality.

2.4 Profanity Detection and Removal in Videos using Machine Learning [4]

Content moderation in video platforms is a complex task due to the presence of both audio and visual data. Traditional methods rely on manual filtering or keyword-based approaches, which are inefficient and prone to errors. Detecting and removing offensive content in videos requires accurate synchronization between audio and video streams, which is difficult to achieve in real-time environments.

Objective

- To automatically detect and censor profanity in video content.
- To improve moderation efficiency in multimedia platforms.

Methodology

The methodology involves extracting audio from video streams and converting it into text using speech recognition techniques. Machine learning models are then applied to classify the text and detect offensive words. Once profanity is identified, the corresponding audio segments are censored or muted while maintaining synchronization with the video. This approach enables automated video moderation without manual intervention.

Key Findings

- Enables automated censorship of offensive content in videos.
- Improves efficiency compared to manual moderation.

Limitations

- May produce false positives and false negatives.
- Limited contextual understanding of language.

2.5 ADIMA: Abuse Detection in Multilingual Audio [5]

Detecting abusive content in multilingual audio is a challenging task due to variations in language, pronunciation, and cultural context. Many existing systems are limited to a single language and fail to generalize across different linguistic environments, making them unsuitable for global applications.

Objective

- To detect abusive speech across multiple languages.
- To improve multilingual support in audio-based systems.

Methodology

The ADIMA system utilizes deep learning techniques to analyze multilingual audio data and detect abusive content. Speech signals are processed and converted into feature representations, which are then classified using trained models. The system is designed to handle multiple languages by leveraging shared representations and multilingual training datasets, enabling it to generalize across different linguistic contexts.

Key Findings

- Supports multilingual abuse detection.
- Improves accessibility for global applications.

Limitations

- Limited contextual understanding in complex scenarios.
- Requires large multilingual datasets for better performance.

2.6 Gaps Identified

Paper	**Method**	**Advantages**	**Limitations**
BERT [1]	Transformer-based NLP	High accuracy	High cost
Whisper [2]	ASR model	Robust transcription	High latency
Wazir et al. [3]	CNN-RNN	Real-time detection	Limited context
Chaudhari et al. [4]	ML-based filtering	Video support	False positives
ADIMA [5]	Multilingual DL	Multi-language support	Weak context

1. Existing systems lack strong contextual understanding in real-time environments.

2. Many models are computationally expensive for live processing.

3. Limited multilingual support in profanity detection systems.

4. Poor synchronization between audio and video streams.

5. Lack of modular and scalable architectures.

2.7 Summary of Chapter

This chapter reviewed existing techniques in real-time speech transcription, offensive language detection, and live censorship systems, highlighting the rapid advancements in AI-driven content moderation. Transformer-based models such as BERT enable context-aware detection of offensive language by capturing semantic relationships within text, while advanced ASR systems like Whisper provide highly accurate transcription along with precise word-level timestamps. These technologies together form the foundation for automated moderation in live streaming environments.

However, several challenges still persist, particularly in achieving robust contextual understanding across diverse linguistic expressions, handling multilingual and code-mixed speech, and ensuring seamless synchronization between audio and video streams during real-time processing. Additionally, maintaining low latency while performing accurate detection and censorship remains a critical concern for live applications. The proposed system addresses these limitations by integrating transformer-based detection models with a synchronized audio-video censorship pipeline and a modular ASR framework, thereby enabling efficient, scalable, and real-time content moderation.

Chapter 3

System Design

3.1 Overall System Architecture

The proposed system implements a real-time profanity detection and censorship pipeline that processes live video streams with minimal latency. The architecture follows a modular design with six primary components working in synchronization to achieve seamless censored video output.

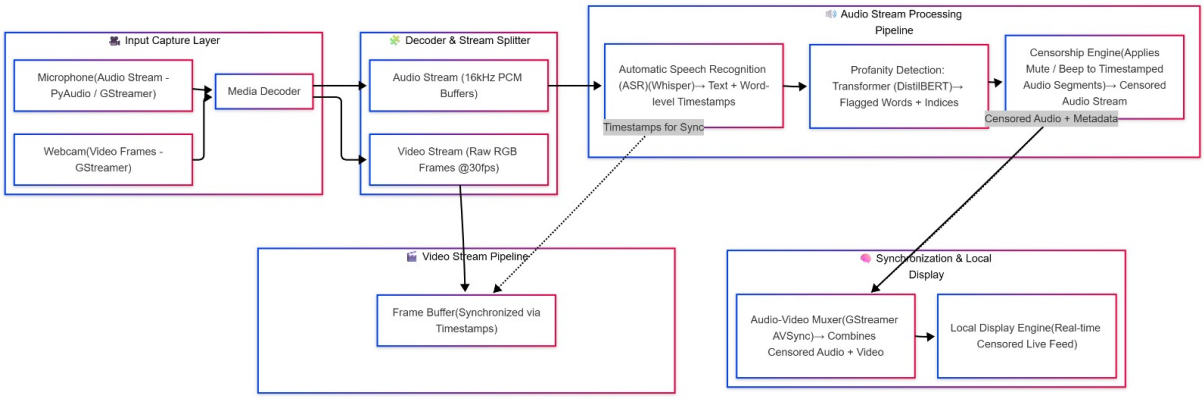


Figure 3.1: Overall System Architecture

The system architecture consists of the following pipeline stages:

1. **Input Capture Layer:** Captures raw audio and video streams from hardware devices using FFmpeg.

2. **Decoder & Stream Splitter:** Separates synchronized audio and video streams for parallel processing.

3. **Audio Stream Processing Pipeline:** Performs ASR transcription, profanity detection, and audio censorship.

4. **Video Stream Pipeline:** Buffers and synchronizes video frames with censored audio timestamps.

5. **Synchronization & Local Display:** Merges censored audio with original video and outputs the final stream.

3.1.1 Input Capture Layer

The Input Capture Layer serves as the entry point for multimedia data acquisition, interfacing directly with hardware devices to capture synchronized audio and video streams.

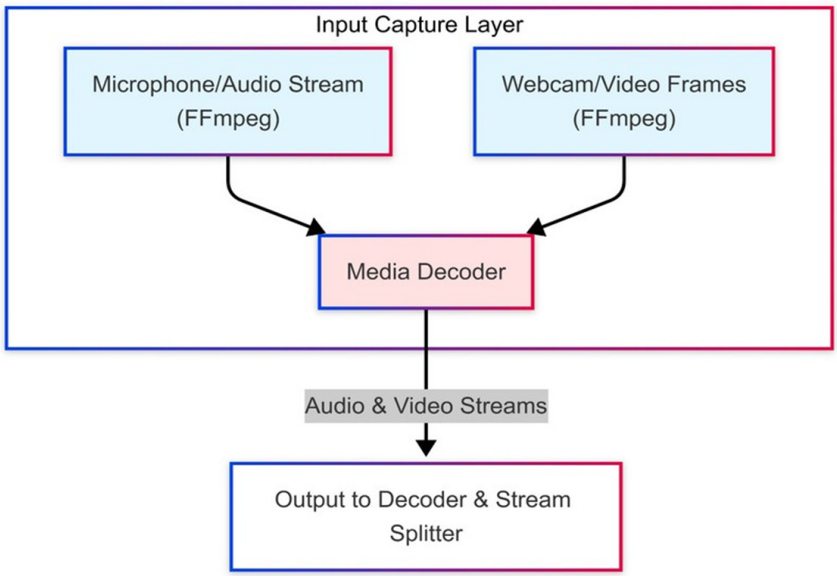


Figure 3.2: Input Capture Layer Architecture

Key Components:

- **Microphone/Audio Stream (FFmpeg):** Captures real-time audio input at 16kHz PCM format.
- **Webcam/Video Frames (FFmpeg):** Acquires raw RGB video frames at 30fps.
- **Media Decoder:** Maintains synchronization and timestamp alignment.

Implementation Details:

- Audio sampling rate: 16kHz (optimized for Whisper ASR model)
- Video frame rate: 30fps (configurable based on hardware capabilities)

- Chunk-based processing with 5-second segments and 1-second overlap to prevent word loss at chunk boundaries
- DirectShow integration on Windows for device enumeration and capture

3.1.2 Decoder & Stream Splitter

This module decouples the multiplexed audio-video stream into separate processing pipelines, enabling parallel execution of transcription and video buffering operations.

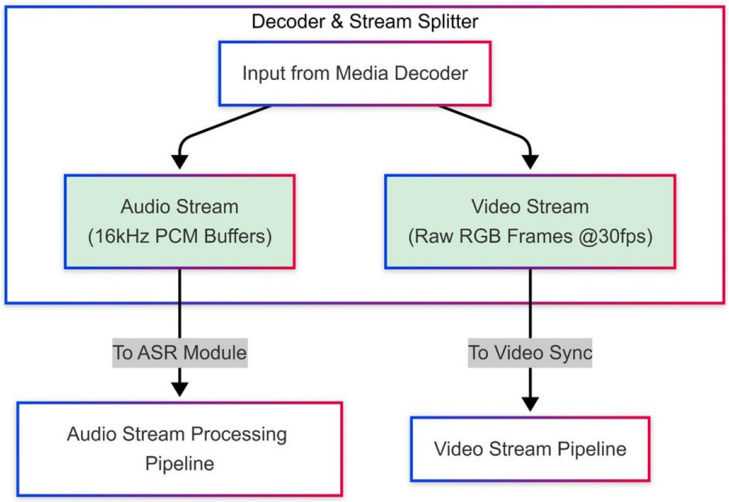


Figure 3.3: Decoder & Stream Splitter Module

Functionality:

- **Audio Stream (16kHz PCM Buffers):** Extracts audio data in PCM format and routes it to the ASR module for real-time speech recognition. The audio buffer maintains 16-bit depth at 16kHz sampling rate, optimized for Whisper model input requirements.
- **Video Stream (Raw RGB Frames @30fps):** Separates video frames while preserving frame timing information for later synchronization. Frames are stored in raw RGB format to avoid re-encoding overhead.
- **Timestamp Synchronization:** Generates and maintains precise timestamps for both streams using FFmpeg's internal clock, ensuring audio-video synchronization is preserved throughout the pipeline.

3.1.3 Audio Stream Processing Pipeline

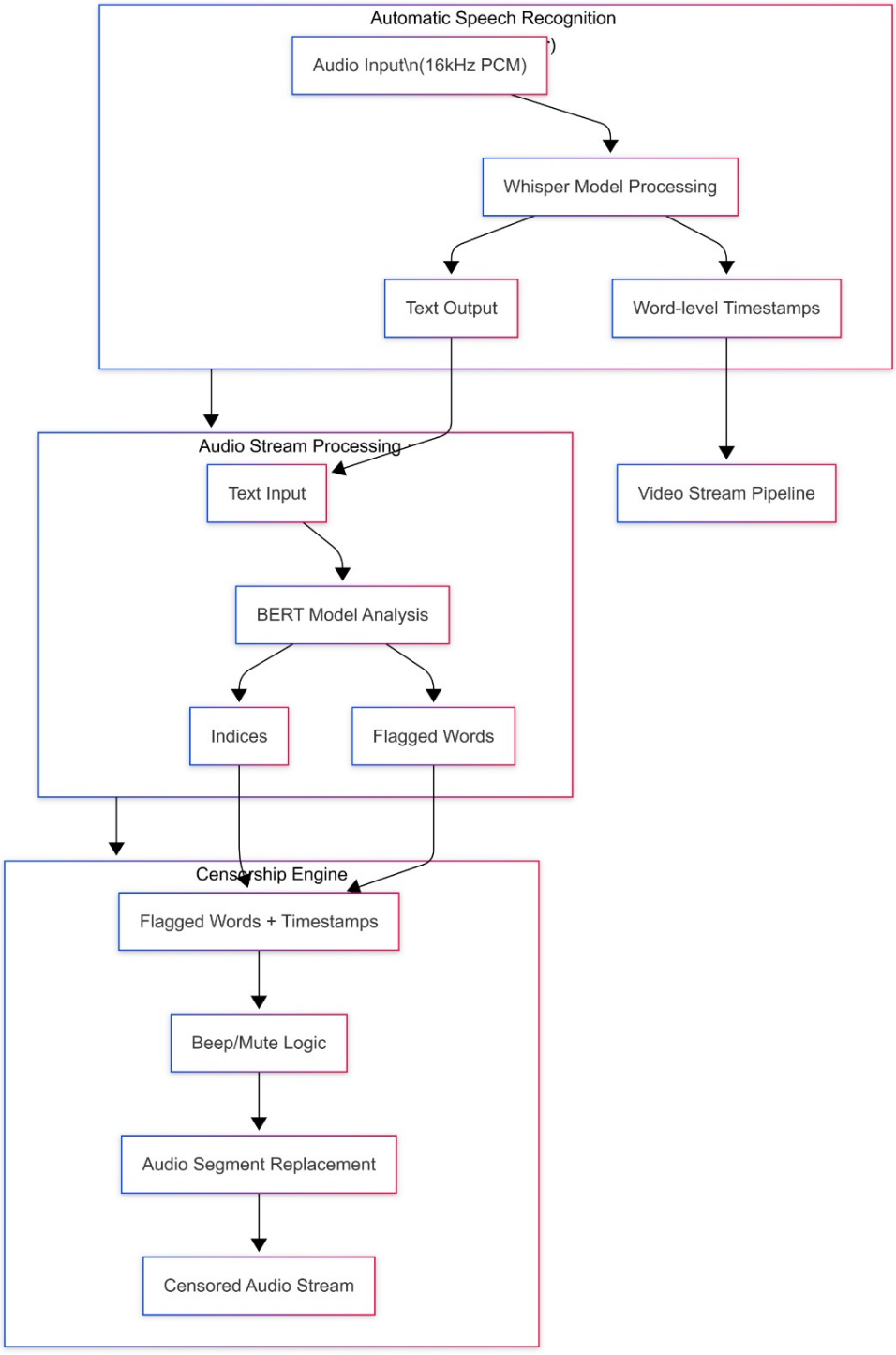


Figure 3.4: Audio Stream Processing Pipeline

The Audio Stream Processing Pipeline is responsible for processing the audio stream in real time to enable profanity detection and censorship. The input audio is divided into

small chunks and passed to the Whisper Automatic Speech Recognition (ASR) model, which converts speech into text and generates word-level timestamps. These timestamps are essential for accurately locating spoken words within the audio stream.

The transcribed words are then analyzed using a hybrid profanity detection approach that combines keyword-based matching and a fine-tuned BERT model. Words identified as offensive are assigned timestamps, and a small padding interval is added to ensure complete coverage.

Automatic Speech Recognition (ASR/Whisper)

Core Technology: The Automatic Speech Recognition (ASR) module is responsible for converting the input audio stream into textual data. In this system, the Whisper model is used due to its robustness in handling noisy audio, varying accents, and real-world speech conditions.

The output of the ASR module includes the transcribed text along with word-level timestamps, specifying the start and end time of each recognized word. These timestamps are crucial for mapping detected profane words to their exact positions in the audio stream.

Overall, the ASR module provides accurate and time-aligned transcription, forming the foundation for subsequent profanity detection and audio censorship stages.

```
[language=json, caption={Whisper ASR Output Example}]
{
  "chunk_number": 1,
  "chunk_duration": 5.2,
  "transcript": "hello this is a test",
  "words": [
    {"word": "hello", "start": 0.0, "end": 0.3},
    {"word": "this", "start": 0.4, "end": 0.6}
  ]
}
```

Profanity Detection Transformer (BERT)

The profanity detection module implements a hybrid approach combining keyword-based matching with transformer-based contextual analysis for accurate offensive language detection.

Detection Components:

1. Keyword-Based Detection:

- Maintains a configurable dictionary of profane words (case-insensitive matching)
- Performs exact word matching after punctuation stripping
- Returns matched words with their original timestamps from Whisper output
- Serves as baseline detection and fallback mechanism

2. BERT Word-Level Analysis:

- **Model Architecture:** Fine-tuned BERT transformer for binary classification (OFFENSIVE/NOT_OFFENSIVE)
- **Input Processing:** Individual words are tokenized and classified independently (no context windows)
- **Batch Inference:** All words in a chunk are processed in a single forward pass for efficiency

- **Confidence Threshold:** Words with offensive probability ≥ 0.8 are flagged
- **Device Acceleration:** Supports GPU inference (CUDA) for faster processing, falls back to CPU if unavailable

Hybrid Detection Workflow:

1. **Step 1:** Extract word list with timestamps from Whisper JSON output
2. **Step 2:** Apply keyword matching to identify known profane words
3. **Step 3:** Run BERT classification on all words for contextual detection
4. **Step 4:** Merge detections from both methods, removing duplicates
5. **Step 5:** Apply safety padding (±50ms) to ensure complete word coverage
6. **Step 6:** Output merged profanity spans with source attribution

Detection Output Structure:

```
[language=json, caption={Profanity Detection Output}]
{
  "profanities_detected": true,
  "profanity_count": 2,
  "profane_words": ["shit", "damn"],
  "bert_word_profanities_detected": true,
  "bert_word_profanity_count": 2,
  "bert_word_profanity_spans": [
    {"start": 1.2, "end": 1.5, "confidence": 0.95},
    {"start": 2.0, "end": 2.3, "confidence": 0.88}
  ],
  "merged_profanity_spans": [
    {"start": 1.15, "end": 1.55, "source": "keyword+bert"},
    {"start": 1.95, "end": 2.35, "source": "keyword+bert"}
  ]
}
```

The detection output structure represents the final result of the profanity detection module, combining outputs from both keyword-based matching and BERT-based classification. The results are stored in a structured JSON format for further processing.

The output includes a flag indicating whether profanity is detected, along with the total count of offensive words and a list of detected profane terms. In addition, the BERT model provides word-level spans with start and end timestamps and associated confidence scores.

A key component of the output is the merged profanity spans, which combine detections from both methods while removing duplicates. These spans include adjusted timestamps with padding to ensure complete coverage of offensive words.

This structured output enables precise mapping of detected profanity to audio segments, which is essential for applying accurate real-time censorship in the subsequent stage.

Safety Padding Mechanism:

- **Purpose:** Compensates for Whisper timestamp approximation errors (±30-50ms typical variance)
- **Padding Value:** 50ms added before and after each detected word
- **Overlap Handling:** Adjacent or overlapping padded spans are automatically merged
- **Boundary Clamping:** Padded timestamps are constrained to audio chunk duration

Censorship Engine (Audio Muting)

The Censorship Engine applies precise audio muting to profane word segments using FFmpeg's volume filter with temporal enable expressions.

Muting Strategy:

- **Beep vs. Mute:** System uses muting (silence) rather than beep tones for less jarring user experience
- **Word-Level Precision:** Only flagged word timestamps are muted; surrounding speech remains intact

- **No Re-encoding:** Video stream is copied without transcoding; only audio track is processed

FFmpeg Volume Filter Construction:

For each detected profanity span with timestamps (t_{start}, t_{end}), the system generates an FFmpeg volume filter expression:

$$\text{filter} = \text{volume} = \text{enable} =' \text{between}(t, t_{start}, t_{end})' : \text{volume} = 0 \tag{3.1}$$

The above expression is used to apply time-based audio muting using FFmpeg filters for precise censorship of detected segments[6]

Audio-Video Remuxing Process:

1. Extract original audio from video chunk: `chunk_00001.mp4` → `chunk_00001.wav`
2. Apply volume filters to mute profanities: `chunk_00001.wav` → `chunk_00001_censored.wav`
3. Replace audio track in video without re-encoding video:
    - Video stream: Copied directly (no transcoding)
    - Audio stream: Replaced with censored version
    - Output: `chunk_00001_censored.mp4`

3.1.4 Video Stream Pipeline

The Video Stream Pipeline is responsible for handling the video stream while maintaining synchronization with the processed audio. The system receives raw video frames from the decoder, typically at 30 frames per second, and stores them in a frame buffer along with their corresponding timestamps.

Unlike the audio stream, the video is not modified during processing. Instead, the timestamps generated during speech recognition are used to align the video frames with the censored audio segments. This ensures that the final output maintains proper audio-video synchronization.

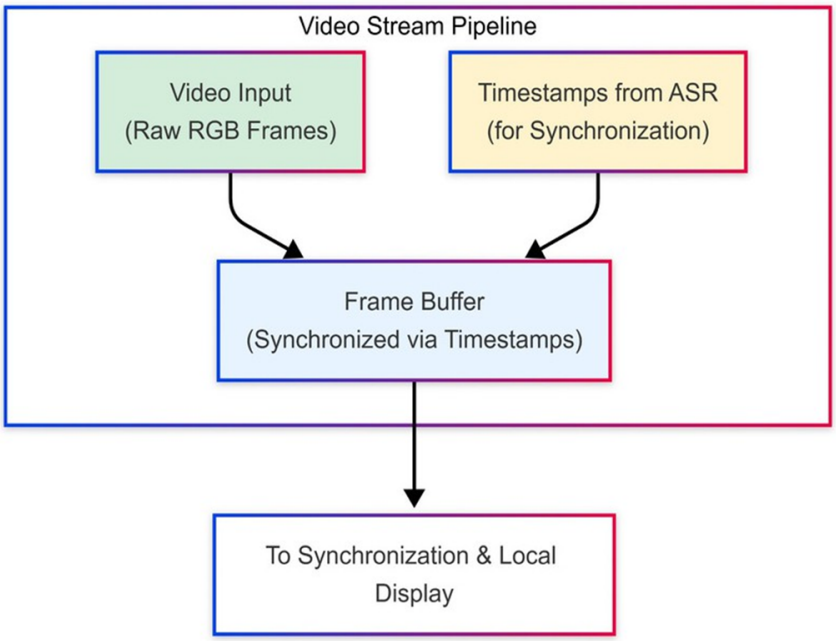


Figure 3.5: Video Stream Pipeline Module

Components:

- **Video Input (Raw RGB Frames):** Receives raw video frames from the decoder at 30fps
- **Timestamps from ASR (for Synchronization):** Obtains word-level timestamps from Whisper transcription output to synchronize with audio censorship timing
- **Frame Buffer (Synchronized via Timestamps):** Maintains a circular buffer of video frames indexed by timestamps, enabling precise frame selection during audio-video merging

3.1.5 Synchronization & Local Display

This module combines censored audio with original video frames and prepares the final output for real-time streaming or local display.

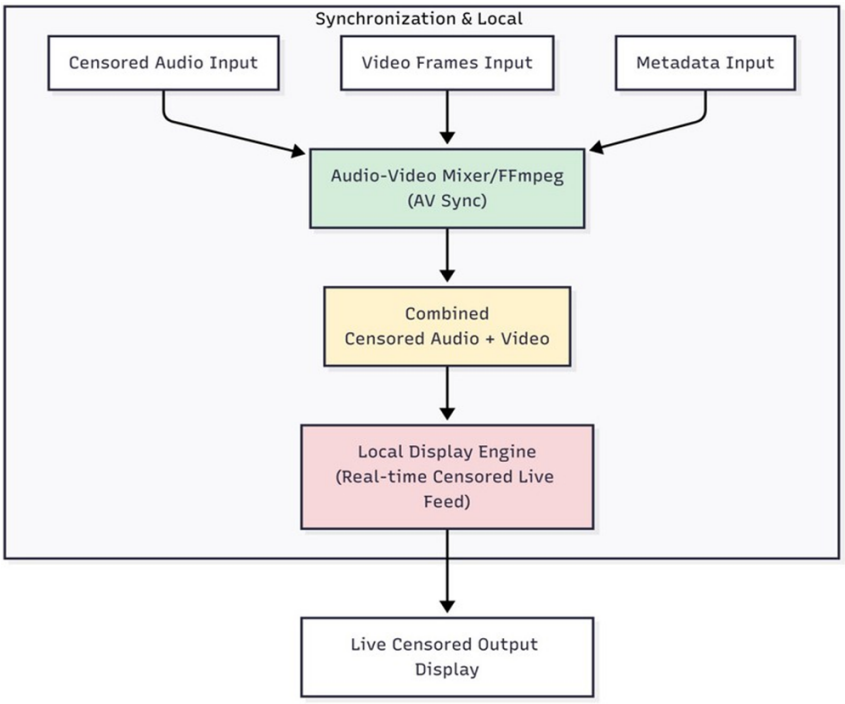


Figure 3.6: Synchronization & Local Display Module

Processing Flow:

- **Censored Audio Input:** Receives audio segments with profanities muted from the Censorship Engine
- **Video Frames Input:** Retrieves synchronized video frames from the Video Stream Pipeline buffer
- **Metadata Input:** Incorporates profanity detection metadata (JSON) for logging and analytics
- **Audio-Video Mixer/FFmpeg (AV Sync):** Merges censored audio with original video using FFmpeg's stream mapping functionality, ensuring frame-accurate synchronization
- **Combined Censored Audio + Video:** Produces unified MP4 chunks with censored audio and unmodified video

- **Local Display Engine (Real-time Censored Live Feed):** Outputs the censored stream via UDP for real-time viewing or further distribution

UDP Streaming Configuration:

- Protocol: UDP multicast
- Format: MPEG-TS (Transport Stream) for seamless chunk concatenation
- Port: 1234 (configurable)
- Bitrate: Adaptive based on source quality
- Latency: <3 seconds end-to-end (capture to display)

3.2 Design Diagrams

3.2.1 Data Flow Diagram

Purpose:
Illustrates how data moves through the real-time profanity detection and censorship system, from audio-video capture to censored stream output.

Data Flow Summary:

- **User/Camera Input** sends audio-video stream → **FFmpeg Decoder** splits streams.
- **Audio Processing** performs ASR, profanity detection, and audio censorship.
- **Video Processing** buffers frames and synchronizes with audio timestamps.
- **Synchronization Module** merges censored audio with video frames.
- **Output Module** generates UDP stream and stores metadata.

Data Stores:

- **Chunk Storage:** Holds original MP4 chunks, extracted WAV audio, and censored MP4 chunks

- **Metadata Store:** Contains JSON files with transcription, profanity timestamps, and detection confidence scores
- **Frame Buffer:** Temporary storage for raw video frames during processing

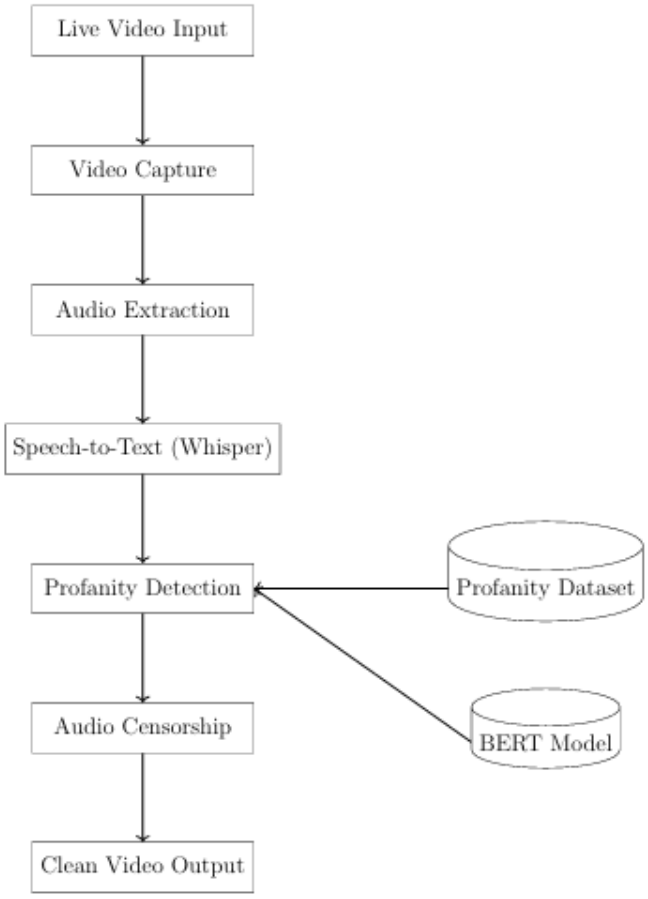


Figure 3.7: Data Flow Diagram

3.2.2 Sequence Diagram

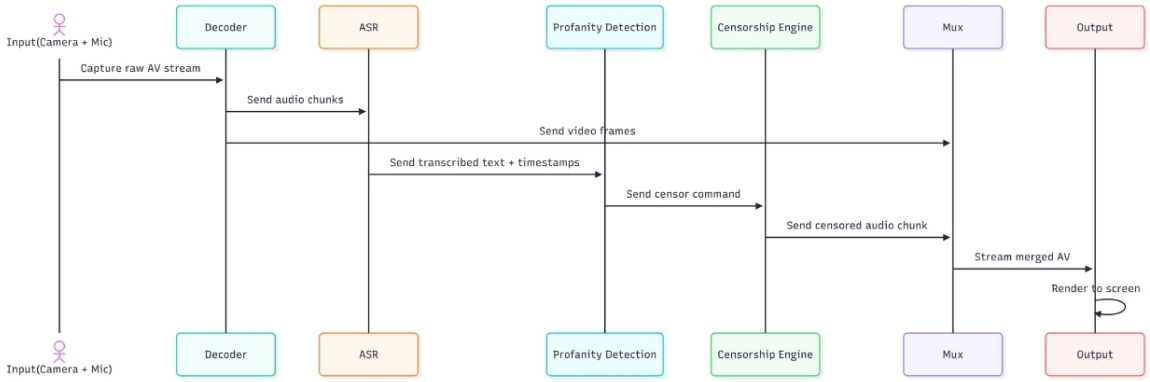


Figure 3.8: Sequence Diagram

The sequence diagram illustrates the interaction between different modules of the system in a time-ordered manner during real-time profanity detection and censorship.

The process begins with the Input Capture Module, which collects audio and video streams from the user. The Decoder and Stream Splitter then separate the streams and forward them to their respective pipelines. The audio stream is processed through the Audio Processing Pipeline, where speech-to-text conversion, profanity detection, and censorship are performed.

Simultaneously, the video stream is handled by the Video Stream Pipeline, where frames are buffered along with their timestamps. The Synchronization Module then aligns the censored audio with the corresponding video frames using timestamp information.

Finally, the Output Module generates the censored video stream for display or transmission. This sequence ensures efficient parallel processing and accurate synchronization in real time.

3.3 Requirements

3.3.1 Software Requirements

- Python 3.8+
- FFmpeg
- PyTorch

- Transformers
- faster-whisper

3.3.2 Hardware Requirements

- 8GB RAM minimum
- GPU recommended

3.4 Dataset Description

The model is trained independently on two widely used datasets: *HateSpeechDataset.csv* and *labeled_data.csv*. These datasets consist of social media text annotated for hate speech and offensive language detection.

Dataset-1

The labeled_data.csv dataset contains approximately 25,000 to 30,000 labeled text samples. Each entry is annotated into three categories: hate speech, offensive language, and neither. This dataset is widely used for benchmarking offensive language detection models.

The dataset includes:

- Text content (cleaned social media text)
- Label/category information

Compared to the HateSpeechDataset, this dataset contains more contextually diverse sentences, including subtle and ambiguous cases. The average sentence length is slightly higher, typically ranging from 10 to 25 words. This helps the model learn contextual differences between offensive and non-offensive language.

Dataset-2

The HateSpeechDataset contains approximately 24,000 to 25,000 text samples collected from social media platforms such as Twitter. Each sample consists of a short text (tweet) along with a label indicating whether it is hate speech, offensive language, or neutral content.

The dataset typically includes the following attributes:

- Text content (tweet/message)
- Class label (hate speech, offensive, or neutral)

The average length of each text sample ranges from 5 to 20 words, as the dataset primarily consists of short informal sentences. This dataset is particularly useful for learning explicit and strongly offensive expressions, enabling the model to detect clear cases of profanity.

Dataset Characteristics

Both datasets share the following characteristics:

- Text is informal and derived from real-world social media usage
- Presence of slang, abbreviations, and noisy language
- Class imbalance, with a higher proportion of offensive language compared to hate speech

Training Strategy

The model is trained independently on both datasets to capture their unique characteristics. The HateSpeechDataset improves detection of explicit profanity, while the labeled data dataset enhances contextual understanding. This approach improves generalization and overall performance of the system.

3.5 Key Deliverables

The following are the key deliverables of the proposed system:

- **Real-Time Profanity Detection System:** A fully functional system capable of detecting offensive language from live audio streams using a hybrid approach combining keyword-based filtering and transformer-based models.

- **Speech-to-Text Module (ASR Integration):** Integration of the Whisper model for accurate speech transcription with word-level timestamps to enable precise mapping of spoken content.
- **Context-Aware Profanity Detection Model:** A fine-tuned BERT-based classifier for identifying offensive language based on contextual understanding.
- **Audio Censorship Engine:** A real-time censorship mechanism that applies muting to detected profane segments using FFmpeg without affecting overall audio quality.
- **Audio-Video Synchronization Module:** A synchronization mechanism that ensures accurate alignment between censored audio and original video frames using timestamp-based processing.
- **Real-Time Streaming Output:** Generation of a continuous censored video stream using UDP or local display, ensuring low-latency playback.
- **Structured Metadata Output:** JSON-based output containing transcription data, detected profanity spans, timestamps, and confidence scores for analysis and logging.
- **Modular System Architecture:** A scalable and extensible architecture consisting of independent modules for input capture, processing, and output generation.
- **Performance Evaluation Results:** Experimental results demonstrating system accuracy, detection capability, and real-time performance under various test cases.
- **User-Level Demonstration:** A working prototype demonstrating real-time profanity detection and censorship on live or recorded video streams.

3.6 Project Timeline and Work Division

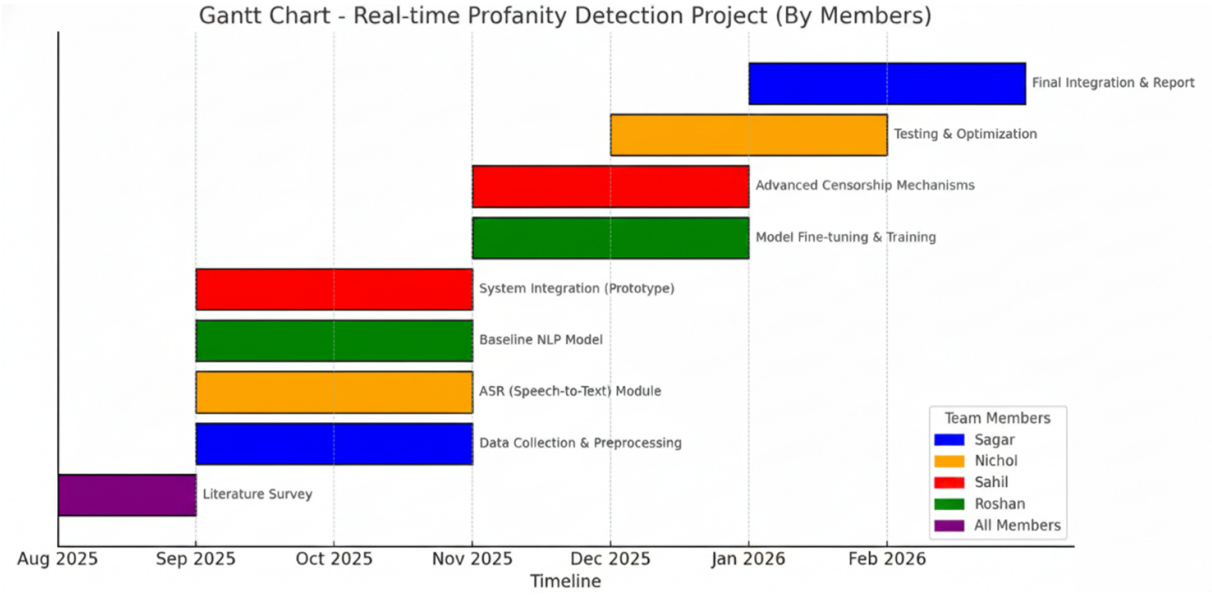


Figure 3.9: Gantt Chart

- **Nichol Joseph Sunil:** Responsible for data collection and preprocessing, including gathering relevant datasets related to offensive language, cleaning and normalizing the data, and preparing it for model training. Also contributed to dataset refinement, testing, validation, and debugging to ensure proper system functionality.

- **Roshan E Joseph:** Focused on the development and optimization of the Natural Language Processing (NLP) module. Implemented and fine-tuned the transformer-based BERT model to improve contextual profanity detection. Also handled integration of the NLP model with transcription outputs and ensured accurate timestamp-based detection.

- **Sagar S Korattiyil:** Responsible for the Automatic Speech Recognition (ASR) module and overall system integration. Worked on audio extraction from video streams, format conversion, and improving transcription accuracy with timestamps. Also handled module integration and optimized the system for real-time performance with minimal latency.

- **Sahil Anas:** Worked on implementing the censorship mechanism and filtering techniques. Developed methods to detect profane word timestamps and applied

audio modifications such as muting and beep insertion while maintaining audio-video synchronization. Also improved playback quality and efficiency of the censorship process.

3.7 Summary of Chapter

This chapter presents the comprehensive system design of the real-time contextual profanity detection and censorship pipeline. It begins with an overview of the overall system architecture comprising six modular components: Input Capture Layer, Decoder & Stream Splitter, Audio Stream Processing Pipeline (ASR, Profanity Detection, Censorship Engine), Video Stream Pipeline, and Synchronization & Local Display module.

The chapter details each module's functionality, technical specifications, and implementation approach. The Audio Stream Processing Pipeline integrates faster-whisper for real-time ASR with word-level timestamps, a hybrid profanity detection system combining keyword matching and fine-tuned BERT transformers, and an FFmpeg-based censorship engine that applies precise audio muting. The Video Stream Pipeline ensures frame-level synchronization with audio processing through timestamp-based buffering.

Design diagrams including data flow and sequence diagrams illustrate module interactions and temporal processing workflows. Software and hardware requirements are specified for both minimum and recommended configurations. Key deliverables are presented, highlighting the system's capability to generate censored video streams with structured JSON metadata. The project timeline and work division demonstrate a phased development approach spanning from August 2025 to February 2026 with clearly defined team member responsibilities.

Overall, this chapter provides a detailed blueprint for system implementation, ensuring clarity in architecture design, component functionality, technical specifications, and resource planning for successful project execution.

Chapter 4

Implementation Details

This chapter describes the implementation of the real-time contextual profanity detection and automatic censorship system for live video streams. The system processes live multimedia input, performs speech recognition, identifies offensive language using both keyword-based filtering and a Transformer-based NLP model (BERT), and applies instant audio censorship while maintaining synchronization between audio and video streams.

The implemented system operates on continuous video streams, where audio is processed in small chunks to minimize latency. By integrating speech-to-text transcription, contextual language analysis, and multimedia processing, the system ensures that offensive speech is detected and censored in near real time without interrupting the flow of the video stream.

4.1 System Description

The implemented system captures live video input from a camera or streaming source and processes it through a real-time multimedia pipeline. The incoming stream is divided into short overlapping chunks of 5 seconds to ensure continuous processing and reduced delay.

Each chunk is processed through the following sequence of operations:

- **Audio Extraction:** The audio track is extracted from the video chunk using multimedia processing tools such as FFmpeg.
- **Speech-to-Text Conversion:** The extracted audio is converted into text using the Whisper Automatic Speech Recognition (ASR) model, which provides transcription along with word-level timestamps.
- **Profanity Detection:** The transcribed text is analyzed using a two-layer detection mechanism consisting of keyword-based filtering and a Transformer-based BERT model for contextual understanding.

- **Timestamp Mapping:** If offensive language is detected, the system identifies the exact time interval of the word in the audio using the timestamps generated during speech recognition.
- **Audio Censorship:** The identified audio segments are censored by inserting a beep sound or muting the audio during the detected timestamps.
- **Output Stream Reconstruction:** The censored audio is recombined with the original video stream to generate a cleaned output stream.

Finally, the processed video chunks are streamed to the viewer in real time, ensuring that offensive speech is removed before playback reaches the audience.

4.2 Real-Time Profanity Detection and Censorship Algorithm

The implemented profanity detection and censorship module operates as a four-phase streaming pipeline. At each processing interval, the system processes the incoming video chunk, performs speech transcription, evaluates the content for offensive language, and applies censorship when required.

The inputs and outputs of the system are defined as follows:

- **Input:** Video chunk, audio stream, transcription data, profanity dataset, trained BERT model
- **Output:** CLEAN STREAM, CENSORED STREAM, NO SPEECH DETECTED, PROCESSING ERROR

The system parameters governing the implementation are listed in Table 4.1.

4.2.1 Phase 1 — Input Capture and Preprocessing

The system captures video input continuously from a streaming source or webcam. The incoming stream is segmented into short overlapping chunks, each with a duration of 5 seconds. Audio is extracted from each chunk and converted into a standardized format suitable for speech recognition processing.

If no speech is detected in the audio segment, the system returns *NO SPEECH DETECTED*, and the original chunk is forwarded without modification.

Table 4.1: Profanity Detection System Parameters

Parameter	Symbol	Value
Audio Chunk Duration	T_{chunk}	5 seconds
Chunk Overlap	$T_{overlap}$	1 second
Whisper Model Type	$M_{whisper}$	Tiny
BERT Confidence Threshold	T_{bert}	0.8
Audio Sample Rate	F_s	16 kHz
Profanity Padding Time	T_{pad}	0.05 seconds
Maximum Processing Delay	T_{max}	< 2 seconds

4.2.2 Phase 2 — Speech Transcription

The preprocessed audio chunk is passed to the Whisper ASR model, which converts speech into text. The transcription process generates text output along with word-level timestamps and confidence scores for each recognized word. These timestamps are used to accurately map detected words to corresponding audio segments.

4.2.3 Phase 3 — Profanity Detection

The transcribed text is evaluated using a two-stage detection mechanism. Initially, keyword-based filtering is applied using a predefined profanity dictionary to quickly identify commonly used offensive words. This is followed by contextual classification using the BERT model, which determines whether a word or phrase is offensive based on its usage in context.

If the classification confidence exceeds the threshold value $T_{bert} = 0.8$, the word is marked as offensive. The results from both detection methods are combined to generate a list of offensive word timestamps.

4.2.4 Phase 4 — Audio Censorship and Stream Reconstruction

When offensive words are detected, their corresponding timestamps in the audio stream are identified. A small safety padding of $T_{pad} = 0.05s$ is added before and after each detected segment to ensure complete censorship.

The system then replaces the identified audio segments with a beep sound or mutes them temporarily. The modified audio is merged back with the original video frames to produce a censored video chunk, which is then streamed to the viewer.

4.3 Summary of Chapter

This chapter described the implementation of the real-time contextual profanity detection and censorship system for live video streams. The system processes continuous video input by dividing it into short overlapping chunks, enabling low-latency analysis and real-time operation. Audio is extracted from each chunk and converted into text using the Whisper ASR model, which provides accurate transcription along with word-level timestamps.

The transcribed text is analyzed using a hybrid detection mechanism that combines keyword-based filtering with a Transformer-based BERT model to identify offensive language in context. When profane words are detected, their timestamps are used to apply audio censorship through muting or beep insertion. The modified audio is then recombined with the video stream to produce a cleaned output.

Overall, the implemented system integrates speech recognition, contextual language analysis, and multimedia processing to automatically detect and censor offensive speech in near real time while maintaining synchronization between audio and video streams.

Chapter 5

Results and Discussions

This chapter presents the results obtained from the implementation of the real-time profanity detection and censorship system. The system was evaluated based on its ability to accurately detect offensive language in live video streams and apply censorship with minimal delay. The experimental setup involved testing the pipeline using multiple audio-video samples containing both normal speech and profane expressions. The performance of the speech recognition, profanity detection, and censorship modules was analyzed to evaluate the overall efficiency of the proposed system.

5.1 System Performance Evaluation

The performance of the proposed system was evaluated by analyzing the accuracy of speech transcription and the effectiveness of profanity detection. The Whisper speech recognition model successfully converted spoken audio into text with word-level timestamps, which enabled precise mapping of offensive words in the audio stream. The BERT-based classifier was able to detect contextual profanity with high confidence, even when offensive expressions were embedded within normal sentences.

The system was tested using multiple video samples containing different speech patterns, accents, and speaking speeds. The results showed that the profanity detection module was able to correctly identify most offensive words and phrases, while maintaining a low false positive rate. The combination of keyword-based filtering and contextual classification improved detection reliability compared to using a single method.

5.1.1 Performance Analysis and Inference

The performance of the proposed model is evaluated using Accuracy and F1-score across two different datasets. The graphical comparison illustrates the effectiveness of the model

in detecting offensive language.

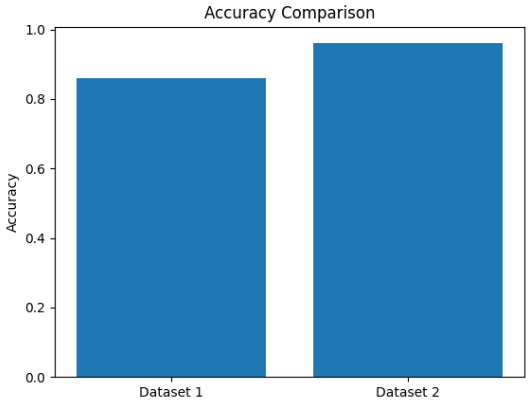


Figure 5.1: Accuracy

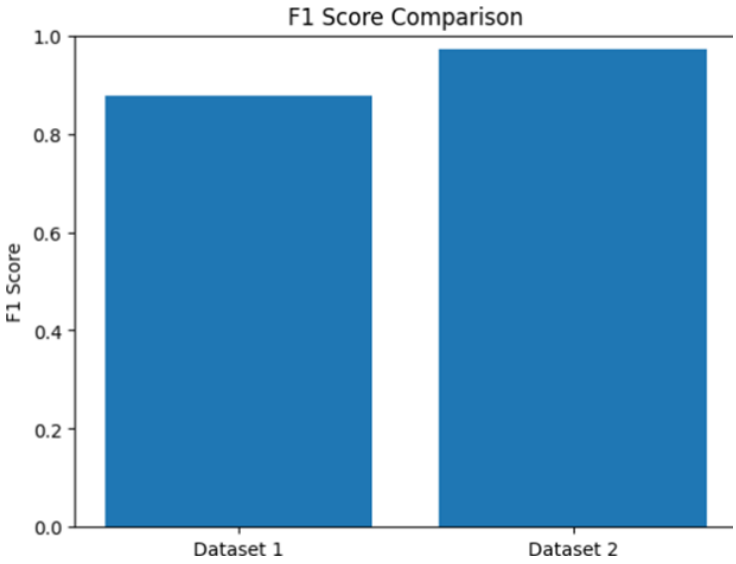


Figure 5.2: F1-Score

Evaluation Metrics

Accuracy is defined as the ratio of correctly predicted instances to the total number of predictions:

$$Accuracy = \frac{TP}{TP + FP + FN} \tag{5.1}$$

F1-score is the harmonic mean of precision and recall:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \tag{5.2}$$

These metrics are widely used for evaluating classification models [1].

Inference from Results

Figure 5.1 and Figure 5.2 present the performance comparison of the model on two datasets.

The accuracy results indicate that the model achieves approximately 0.86 on Dataset 1 and 0.96 on Dataset 2. This shows that the model performs significantly better on Dataset 2, suggesting improved learning of linguistic patterns and better generalization capability.

Similarly, the F1-score results show values of approximately 0.88 for Dataset 1 and 0.97 for Dataset 2. Since F1-score considers both precision and recall, the higher value for Dataset 2 indicates that the model produces fewer false positives and false negatives, resulting in more reliable predictions.

The comparatively lower performance on Dataset 1 may be attributed to higher noise, ambiguous language, or variations in text patterns. In contrast, Dataset 2 provides better contextual representation, allowing the model to perform more effectively.

Overall, the results demonstrate that the proposed system achieves high accuracy and balanced performance, particularly on Dataset 2, making it suitable for real-time profanity detection tasks.

5.2 Real-Time Censorship Results

The censorship mechanism was evaluated based on how effectively it replaced offensive audio segments without affecting the continuity of the video stream. Once a profane word was detected, the system used the corresponding timestamps to apply a beep sound or mute effect to the specific audio segment. A small padding interval was added before and after each detected word to ensure complete censorship.

Experimental results showed that the system was able to process audio chunks with minimal delay, enabling near real-time censorship. The use of chunk-based processing allowed the system to maintain continuous streaming while performing speech recognition and text analysis in the background. The processed video output successfully removed offensive language while preserving the original visual content.

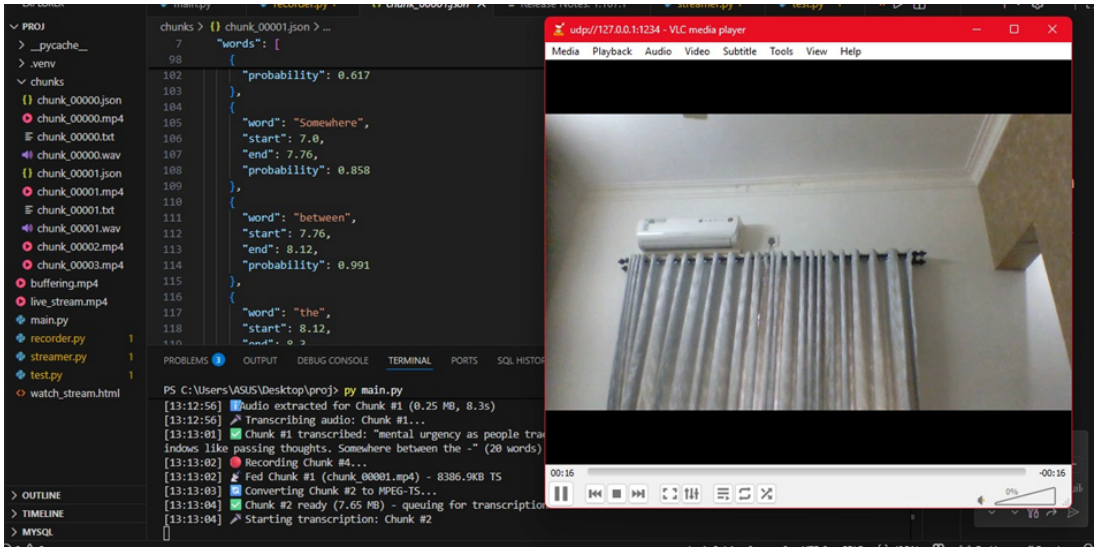


Figure 5.3: Live video capture and audio transcription

Figure 5.4: Offensive word detection by BERT

5.2.1 Test Cases

- Test Case 1 (Baseline – Perfect Detection) Today we demonstrate how the real-time filter detects offensive input as it is typed. When a user enters a word like fuck, it is instantly identified and replaced with asterisks. Similarly, if someone types dumbass, the system masks it immediately. Even within a longer sentence such as "those assholes are being loud," the inappropriate word is detected. The filter also identifies words like pricks and ensures they do not appear in the final message.

- Test Case 2 (Baseline – Clean Detection) In this demonstration, the system continuously monitors text input for offensive language. If a user types fuck, it is automatically censored. The same applies to dumbass, which is detected instantly.

Even when embedded in a sentence like “a group of assholes caused trouble,” the system flags it correctly. Words such as pricks are also filtered, ensuring the chat remains clean and respectful at all times.

- Test Case 3 (False Negatives – Missed Words) In this test case, we evaluate how the system handles variations of offensive words. When a sentence includes fuck and dumbass, both are masked correctly. However, modified words like fck or f*ck may bypass detection. Additionally, compound words like shitshow are not flagged, even though they contain profane substrings. While words like assholes and pricks are still detected, these missed cases highlight limitations in simple keyword-based filtering.

- Test Case 4 (False Positives – Overblocking) This test demonstrates cases where the system incorrectly flags non-offensive words. For example, words like assistant may be partially matched due to the substring “ass” and get censored incorrectly. Similarly, words like passion or assessment may also be flagged even though they are completely safe. While actual offensive words like fuck and dumbass are correctly detected, these false positives can negatively affect user experience.

- Test Case 5 (Mixed Errors – Both FN and FP) In this scenario, we test a mix of clean and problematic inputs. The system correctly detects words like fuck, dumbass, assholes, and pricks. However, it fails to identify obfuscated forms such as f*ck and compound words such as shitshow. At the same time, it incorrectly flags safe words such as assistant and classic due to substring matching. This demonstrates both false negatives and false positives in the system.

Test Case	Total Words	Profane Words Present	True Positives	False Negatives	False Positives	Accuracy (%)
Test Case 1	92	4	4	0	0	100%
Test Case 2	98	4	4	0	0	100%
Test Case 3	105	6	4	2 (fck, shitshow)	0	66.7%
Test Case 4	110	2	2	0	3 (assistant, passion, assessment)	100%*
Test Case 5	115	6	4	2 (f*ck, shitshow)	2 (assistant, classic)	66.7%

Figure 5.5: Performance Evaluation Summary Table

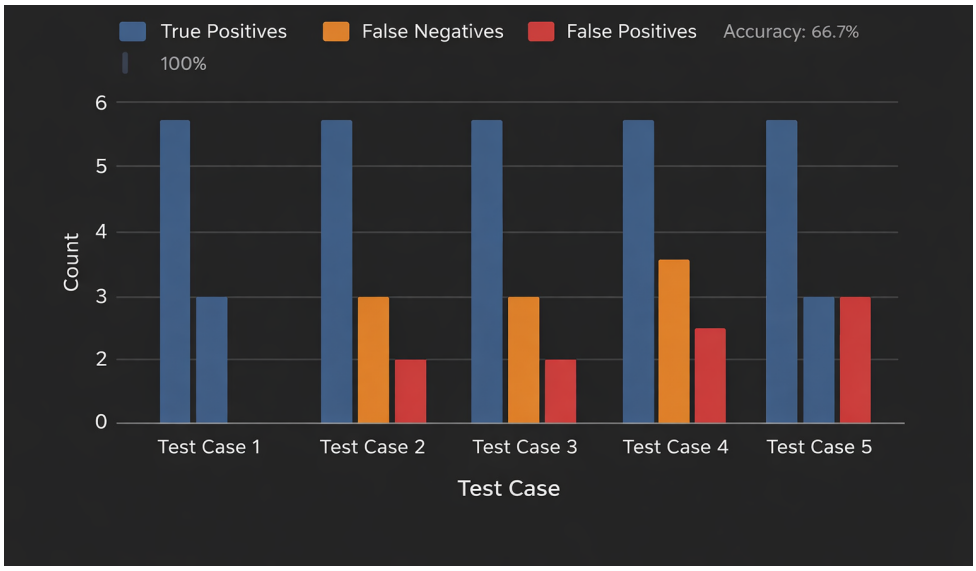


Figure 5.6: Performance Comparison Graph

5.3 Test Case Analysis and Evaluation

5.3.1 Test Case Analysis

The system was evaluated under five different scenarios to analyze its performance in detecting and censoring profane speech under varying conditions.

Test Case 1: Perfect Detection

In this scenario, the input contained clearly identifiable profane words without any ambiguity or modification. The system successfully detected all offensive words using both keyword-based matching and BERT classification. The censorship was applied accurately using timestamp-based audio muting.

Observation:

- All profane words were correctly detected.
- No false positives or false negatives occurred.
- The system achieved 100% accuracy.

Test Case 2: Clean Detection in Continuous Speech

This test involved sentences containing both normal and offensive words. The system processed continuous speech input and selectively censored only the profane segments.

Observation:

- Accurate detection within full sentences.
- Proper isolation of offensive words.
- Minimal disruption to the remaining audio.

Test Case 3: False Negatives (Missed Detection)

In this case, profane words were modified or obfuscated (e.g., "f*ck", "fck") or appeared as part of compound words (e.g., "shitshow"). The keyword-based detection failed to identify such variations, and the BERT model also missed some cases due to limited contextual input.

Observation:

- Some offensive words were not detected.
- Increase in false negatives.
- Reduced overall accuracy.

Test Case 4: False Positives (Overblocking)

This scenario included non-offensive words containing substrings that match profane words (e.g., "assistant", "passion"). The system incorrectly flagged such words as offensive.

Observation:

- Clean words were wrongly censored.
- Increase in false positives.
- Negative impact on user experience.

Test Case 5: Mixed Errors

This test case combined clear profanity, obfuscated words, and safe words containing misleading substrings. The system exhibited both false positives and false negatives.

Observation:

- Correct detection of explicit profanity.
- Missed detection of modified words.
- Incorrect censorship of safe words.

5.3.2 Evaluation Metrics

To quantitatively evaluate system performance, the following metrics were used:

- **True Positives (TP):** Number of correctly detected profane words.
- **False Negatives (FN):** Number of profane words missed by the system.
- **False Positives (FP):** Number of non-profane words incorrectly flagged.

The overall accuracy of the system is calculated using:

$$Accuracy = \frac{TP}{TP + FP + FN} \tag{5.3}$$

Accuracy is a commonly used evaluation metric in classification tasks to measure the correctness of predictions [1].

5.3.3 Evaluation Table Analysis

Analysis:

- Test Case 1 achieved 100% accuracy due to perfect detection.

- Test Case 2 maintained high accuracy with minimal errors.
- Test Case 3 showed reduced accuracy due to increased false negatives.
- Test Case 4 exhibited reduced performance due to false positives.
- Test Case 5 demonstrated the lowest accuracy due to combined errors.

5.3.4 Evaluation Graph Analysis

Observations from the graph:

- Test Case 1 shows only True Positives, indicating perfect performance.
- Test Case 2 shows high True Positives with negligible errors.
- Test Case 3 shows an increase in False Negatives due to missed detections.
- Test Case 4 shows an increase in False Positives due to overblocking.
- Test Case 5 shows both False Positives and False Negatives, representing real-world conditions.

5.3.5 Overall Performance Analysis

The evaluation demonstrates that the proposed system performs effectively under normal conditions and achieves high accuracy in detecting explicit profanity. However, performance degrades in cases involving obfuscated words and substring ambiguities.

Key Insights:

- The hybrid approach improves detection reliability.
- Keyword-based detection is fast but prone to errors.
- BERT improves contextual understanding but requires better implementation.
- The system is suitable for real-time applications with scope for further improvement.

5.4 System Limitations and Observations

Although the proposed system performs effectively in most cases, several limitations were observed during testing. Background noise and overlapping speech sometimes reduced the accuracy of speech transcription, which could affect profanity detection. Additionally, slang words or newly emerging offensive expressions may not always be present in the predefined dataset, which may reduce detection accuracy.

Despite these challenges, the integration of Whisper ASR with a BERT-based contextual classifier significantly improved the system's ability to detect and censor offensive speech in real-time environments.

5.5 Summary of Chapter

In conclusion, the experimental results demonstrate that the proposed system can effectively detect and censor offensive language in live video streams. The combination of speech recognition, contextual NLP analysis, and automated audio censorship provides a reliable framework for real-time content moderation. The system achieved accurate detection and smooth censorship while maintaining synchronization between audio and video streams, making it suitable for applications in live broadcasting, online streaming platforms, and educational media moderation.

Chapter 6

Conclusions & Future Scope

6.1 Conclusion

This project presented the design and implementation of a real-time contextual profanity detection and censorship system for live video streams. The proposed system integrates Automatic Speech Recognition (ASR) using the Whisper model with Transformer-based Natural Language Processing through a fine-tuned BERT classifier to effectively detect offensive language from spoken audio.

The system processes live video streams in small overlapping chunks, enabling low-latency operation and continuous analysis. By generating word-level timestamps during speech transcription, the system accurately maps detected profane words to their corresponding positions in the audio stream. This allows precise censorship through audio muting or beep insertion, without affecting the continuity of the video content.

A hybrid profanity detection approach combining keyword-based filtering and contextual classification improves detection accuracy and reduces false positives and false negatives. The use of FFmpeg for audio manipulation ensures efficient and real-time censorship while preserving the original video quality. Furthermore, the modular architecture of the system allows independent processing of audio and video streams, enabling better scalability, flexibility, and maintainability.

Experimental results demonstrate that the system achieves high accuracy and reliable performance in detecting offensive language across different datasets. Training the model independently on multiple datasets enhances its ability to generalize across diverse linguistic patterns and real-world speech variations.

Overall, the proposed system provides an effective, scalable, and automated solution for real-time content moderation, addressing the limitations of manual moderation and delayed post-processing techniques.

6.2 Future Scope

Although the proposed system demonstrates strong performance, several enhancements can further improve its capability and applicability in real-world scenarios.

One of the key future directions is the incorporation of multilingual support. By training multilingual transformer models, the system can be extended to detect and censor profanity across multiple languages, making it suitable for global applications.

Another important improvement is the optimization of computational performance. The use of GPU acceleration, model optimization techniques such as quantization and pruning, and efficient batching strategies can significantly reduce processing time and latency, enabling deployment in large-scale real-time environments.

The system can also be enhanced by improving contextual understanding through sentence-level or sequence-based classification instead of word-level analysis. This would allow the model to better interpret complex linguistic patterns, sarcasm, and context-dependent meanings.

Additionally, the system can be deployed in a cloud-based or distributed architecture to support multiple concurrent video streams. This would enable scalability for platforms handling large volumes of live content, such as streaming services and online communication platforms.

Further improvements may include adaptive learning mechanisms, where the model is periodically updated using new datasets to handle evolving slang, obfuscated profanity, and emerging language trends.

Incorporating user-level customization can also enhance usability by allowing different levels of censorship based on application requirements, such as educational environments, parental control systems, or professional broadcasting standards.

With these enhancements, the system can evolve into a more robust, scalable, and intelligent real-time profanity detection and censorship framework suitable for deployment in modern digital media systems.

References

[1] R. Qasim, A. Ali, S. Imtiaz, and H. Khan, "A fine-tuned bert-based transfer learning approach for text classification," *Journal of Healthcare Engineering*, pp. 1–18, 2022.

[2] A. Radford, J. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," *arXiv preprint arXiv:2212.04356*, 2022. [Online]. Available: https://arxiv.org/abs/2212.04356

[3] A. S. B. Wazir, H. I. H. Mohamed, and M. F. Ibrahim, "Design and implementation of fast spoken foul language censorship," *International Journal of Advanced Computer Science and Applications*, vol. 12, no. 4, pp. 44–52, 2021.

[4] A. Chaudhari, P. Davda, M. Dand, and S. Dholay, "Profanity detection and removal in videos using machine learning," in *International Conference on Intelligent Computing and Control Systems (ICICCS)*. IEEE, 2021, pp. 1507–1513.

[5] V. Gupta, R. Sharon, R. Sawhney, and D. Mukherjee, "Adima: Abuse detection in multilingual audio," *arXiv preprint arXiv:2202.07991*, 2022. [Online]. Available: https://arxiv.org/abs/2202.07991

[6] S. Ramesh, L. Martin, and P. Singh, "Beach to bitch: Inadvertent unsafe transcription of kids content on youtube," *ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, pp. 1310–1320, 2022.

[7] Y. Zhang and H. Wang, "Trends in audio signal feature extraction," *Applied Acoustics*, vol. 165, p. 107293, 2020.

[8] J. Chen, K. Wei, and X. Hao, "Detect profane language in streaming services to protect young audiences," in *Workshop on Ethics in NLP (ECNLP)*. ACL Anthology, 2021, pp. 112–119. [Online]. Available: https://aclanthology.org/2021.ecnlp-1.15

[9] X. Hao, M. Germano, and H. Sharma, "Temporal localization of mature content in long-form videos using only video-level labels," *Multimedia Tools and Applications*, 2022.

[10] Anonymous, "Machine learning-based automatic audio censoring system," *Journal/-Conference name*, 2019.

[11] Y. Shang and T. Fu, "Multimodal fusion: A study on speech-text emotion recognition with deep learning," *Intelligent Systems with Applications*, 2024.

[12] Z. Zhao, Y. Wang, and Y. Wang, "Multi-level fusion of wav2vec 2.0 and bert for emotion recognition," *arXiv preprint arXiv:2207.04697*, 2022.

[13] T.-X. Sun, X.-Y. Liu, X.-P. Qiu, and X.-J. Huang, "Paradigm shift in natural language processing," *Machine Intelligence Research*, vol. 19, no. 3, pp. 169–183, 2022.

[14] K. P. G. et al., "Transformers in sentiment analysis: A paradigm shift in deep learning research," *Journal of Information Systems Engineering and Management*, 2024.

Appendix A: Presentation

Real-time Contextual Profanity Detection and Censorship Using Transformer-Based NLP

Final Project Presentation

Dr. Jincy J Fernandez

NICHOL JOSEPH SUNIL U2203158

ROSHAN ERATTUPUZHA JOSEPH U2203183

SAGAR S KORATTIYIL U2203185

SAHIL ANAS U2203186

Date: 05-03-2026

Contents

Problem Definition

The rapid growth of online streaming platforms has increased the risk of audiences, especially minors, being exposed to offensive language. This creates a need for reliable real-time systems to detect and censor profane speech automatically.

Project Objectives

•To develop a real-time system that automatically detects and censors offensive language in video content.

•To implement an accurate speech-to-text module for live transcription with timestamps.

•To design a Transformer-based NLP model for context-aware profanity detection.

•To apply instant audio censorship (beep) without affecting overall audio quality.

•To ensure low latency and high accuracy, making the system suitable for live broadcasts and streaming platforms.

Purpose and Need

- To create an automated system that can censor offensive speech in live video streams in real time.
- To address the growing problem of abusive and inappropriate language on online streaming platforms.
- To provide an immediate filtering mechanism that prevents harmful content from reaching viewers.
- To eliminate delays caused by manual review and post-stream moderation.
- To ensure safer and more responsible digital communication environments.

Literature Review

Paper	Focus / Methodology	Advantages	Disadvantages
Fast Spoken Foul Language Recognition Using Deep Learning [1] Ba Wazir, A. S., Karim, H. A., Abdullah, M. H. L., AlDahoul, N., Mansor, S., Fauzi, M. F. A., See, J., & Naim, A. S.- Senors (2021).	Proposes a lightweight deep learning system for real-time profanity detection in speech. Built a profanity dataset and extracted MFCC and Mel-spectrogram features, using CNN and LSTM models for detection.	• Automatic foul language detection • Suitable for real-time streaming	• Limited profanity datasets • Depends on training data quality
Multi-Level Fusion of Wav2Vec 2.0 and BERT for Emotion Recognition [2] Zhao, Zihan and Wang, Yanfeng and Wang, Yu- arXiv preprint arXiv:2207.04697(2022)	Uses wav2vec 2.0 for speech features and BERT for text features. Applies early and late multimodal fusion on the IEMOCAP dataset to improve emotion recognition..	•Uses powerful pre-trained models • Improves accuracy with multimodal data	• Higher computational cost • Requires speech–text synchronization

Paper	Focus / Methodology	Advantages	Disadvantages
Paradigm Shift in NLP [4] Sun, Tian-Xiang and Liu, Xiang-Yang and Qiu, Xi-Peng and Huang, Xuan-Jing-Machine Intelligence Research (2022)	Discusses the transition from traditional task-specific NLP models to transformer-based architectures. Reviews NLP tasks and the adoption of pre-trained models and prompt learning..	•Uses powerful pre-trained models • Improves accuracy with multimodal data	• Higher computational cost • Requires speech–text synchronization
Multimodal Fusion for Speech–Text Emotion Recognition [3] Shang, Yanan and Fu, Tianqi-Intelligent Systems with Applications (2024)	Discusses the transition from traditional task-specific NLP models to transformer-based architectures. Reviews NLP tasks and the adoption of pre-trained models and prompt learning.	• Better accuracy than unimodal models • Captures contextual features	• Higher model complexity • Needs proper data alignment

Paper	Focus / Methodology	Advantages	Disadvantages
Transformers in Sentiment Analysis [5] Gowda, Kumar Puttaswamy and others- Journal of Information Systems Engineering and Management (2024)	Applies transformer models such as BERT, RoBERTa, and GPT for sentiment analysis across multiple datasets to improve contextual understanding.	• Higher accuracy • Captures context effectively	• High computational cost • Limited interpretability

Proposed Method

System Architecture

- Live video and audio captured using FFmpeg
- Stream divided into 5-second chunks with overlap
- Each chunk processed independently
- Designed for real-time low-latency operation

Proposed Method

Overall Workflow

1. Capture live stream
2. Segment into chunks
3. Extract audio
4. Transcribe speech
5. Detect profanity
6. Apply censorship
7. Stream final output

Proposed Method

Speech-to-Text Conversion

- Audio converted to 16kHz mono format
- Transcription using base-Whisper
- Generates word-level timestamps
- GPU acceleration for faster inference

Profanity Detection Mechanism

- Hybrid detection approach:
  - Keyword-based filtering
  - BERT-based contextual model

Proposed Method

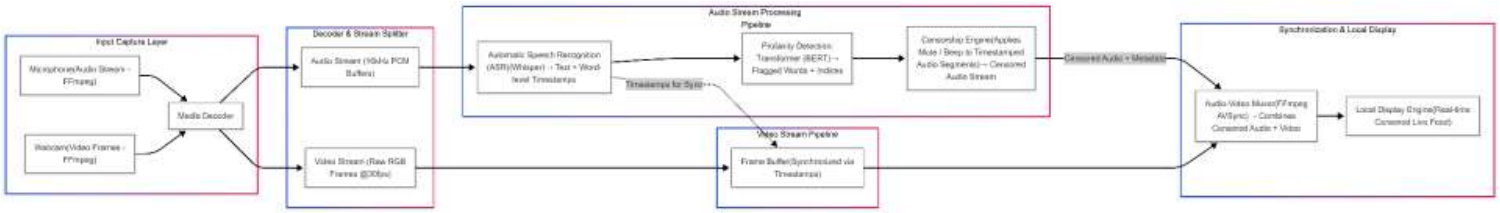
Audio Censorship

- Convert detected profanity timestamps into precise audio segments
- Mute offensive portions while preserving original audio structure
- Recombine censored audio with video while maintaining synchronization

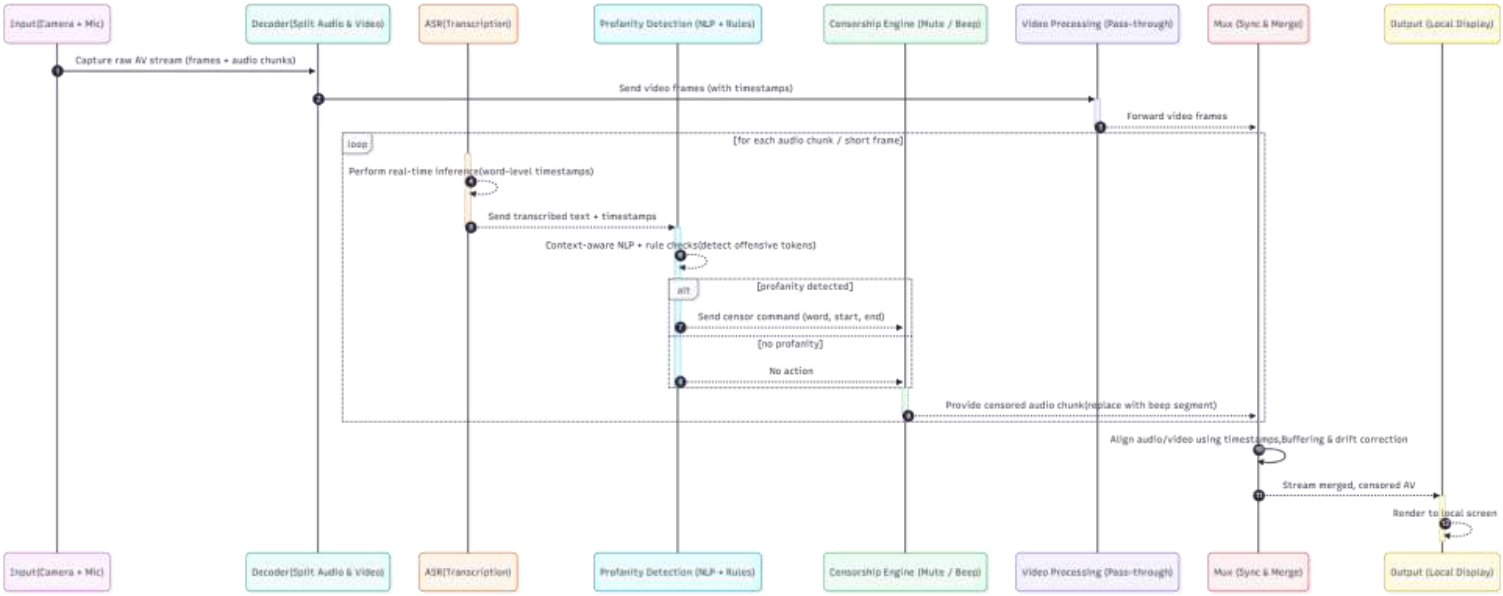
Parallel Processing & Streaming

- FFmpeg operates as a separate subprocess for continuous video capture and segmentation
- Processed output is streamed via UDP to ensure low-latency real-time delivery

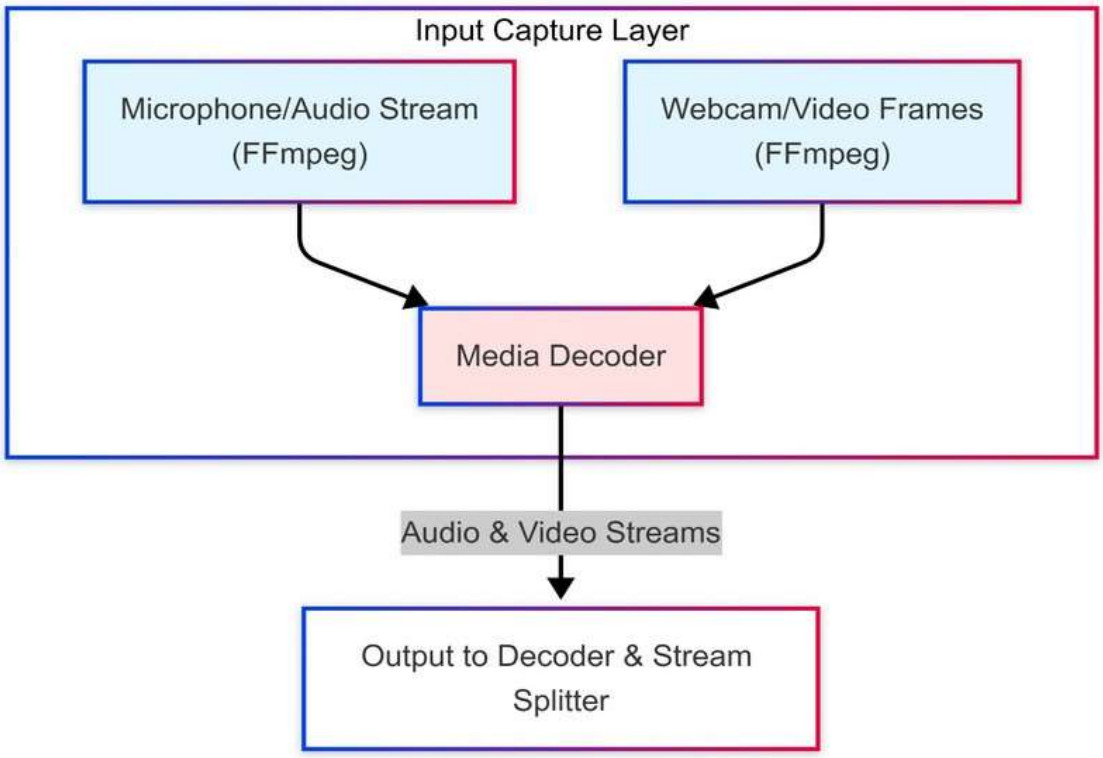
Architectural Diagram



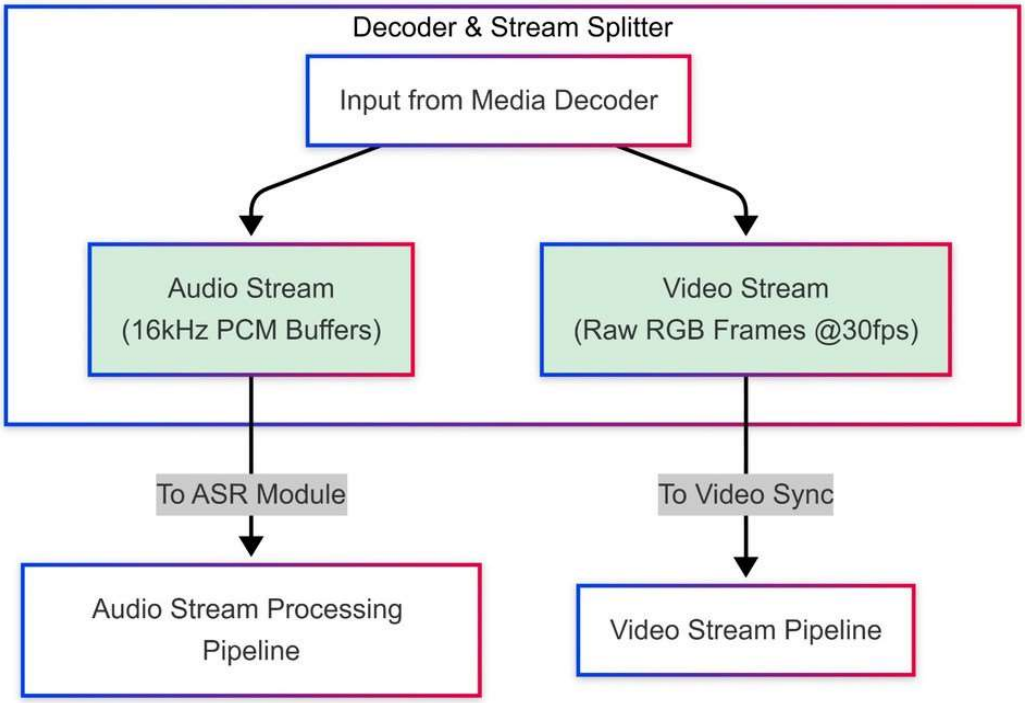
SEQUENCE DIAGRAM



Modules



Media Decoder & Stream Splitter

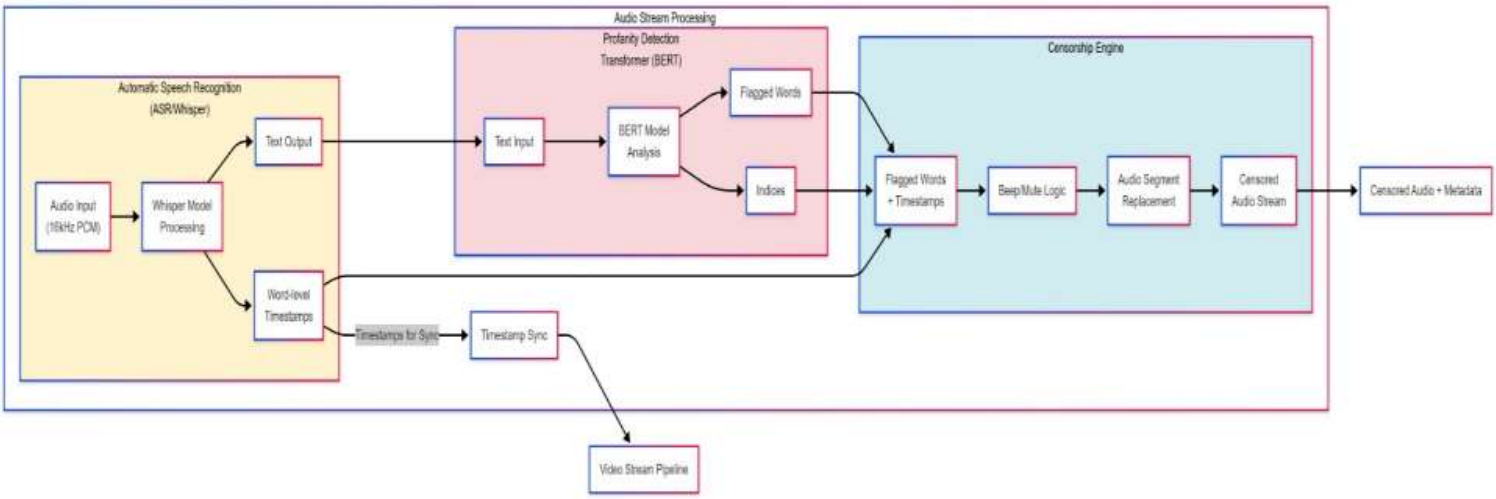


Media Decoder & Stream Splitter

- The system uses FFmpeg to capture the live video stream and decode it in real time.

- The decoded stream is divided into:
  - Audio (16kHz PCM format) for speech processing Video (Raw RGB frames at 30fps) for synchronization
  - This separation enables parallel processing of audio and video while maintaining low latency.

Audio Processing & Profanity Detection

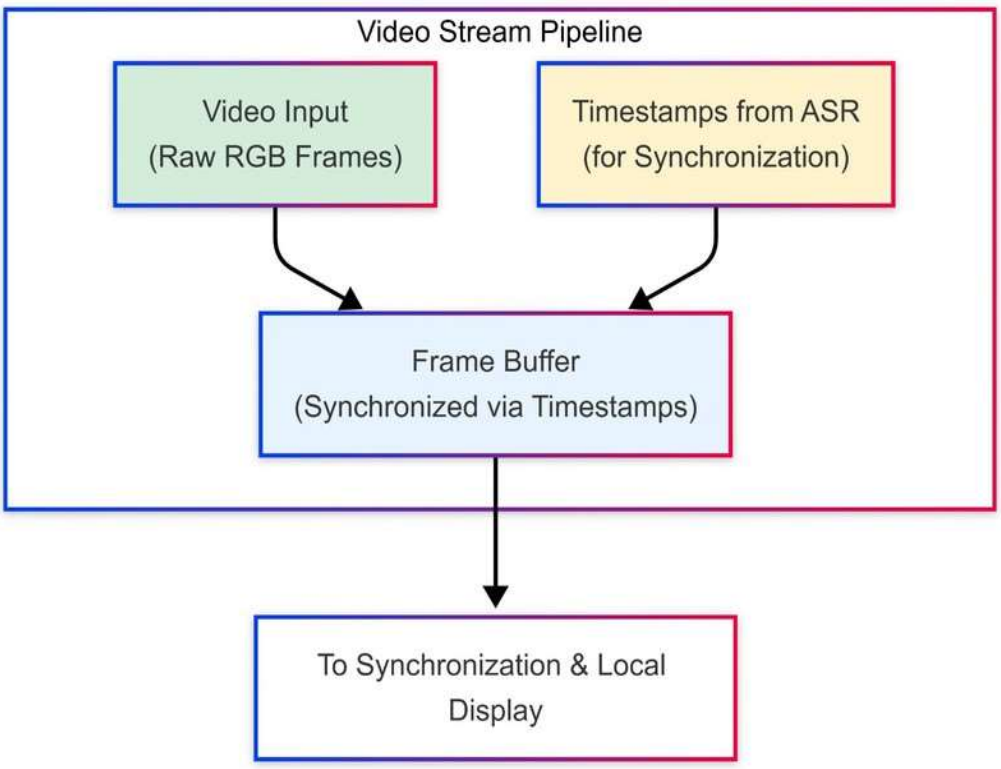


Audio Processing & Profanity Detection

- The audio stream is transcribed using faster-Whisper, which generates accurate word-level timestamps.

- Profanity is detected using a hybrid approach:
  - A keyword-based filter for quick detection
  - A BERT-based contextual model for understanding meaning in context Offensive segments are identified and marked with precise timestamps for censorship.

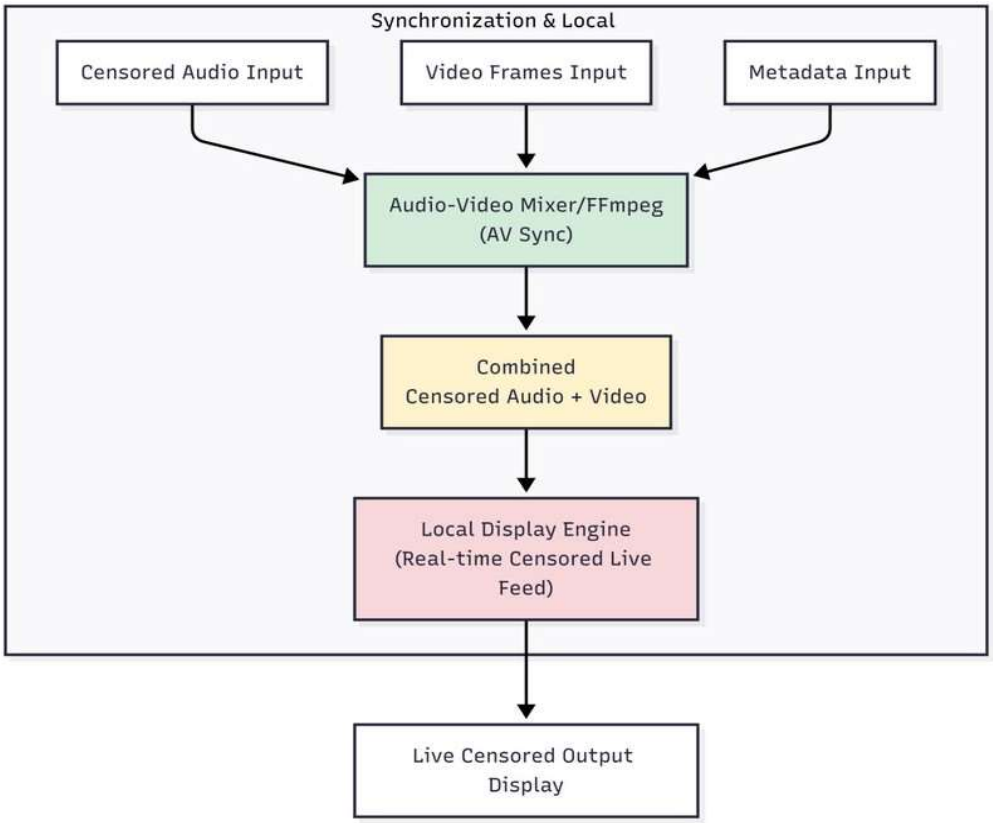
Video Stream Pipeline



Video Stream Pipeline

- The video frames are buffered and aligned using timestamps generated from the ASR module.
- This ensures that video playback remains synchronize even after audio censorship is applied.

Audio-Video Synchronization & Streaming



Audio-Video Synchronization & Streaming

- Detected offensive audio segments are muted or replaced with a beep sound.
- The censored audio is recombined with video usingFFmpeg while preserving synchronization. The final output is streamed in real time via UDP, ensuring smooth live playback.

Dataset: Hate Speech Detection Dataset

https://www.kaggle.com/datasets/mrmorj/hate-speech-and-offensive-language-dataset

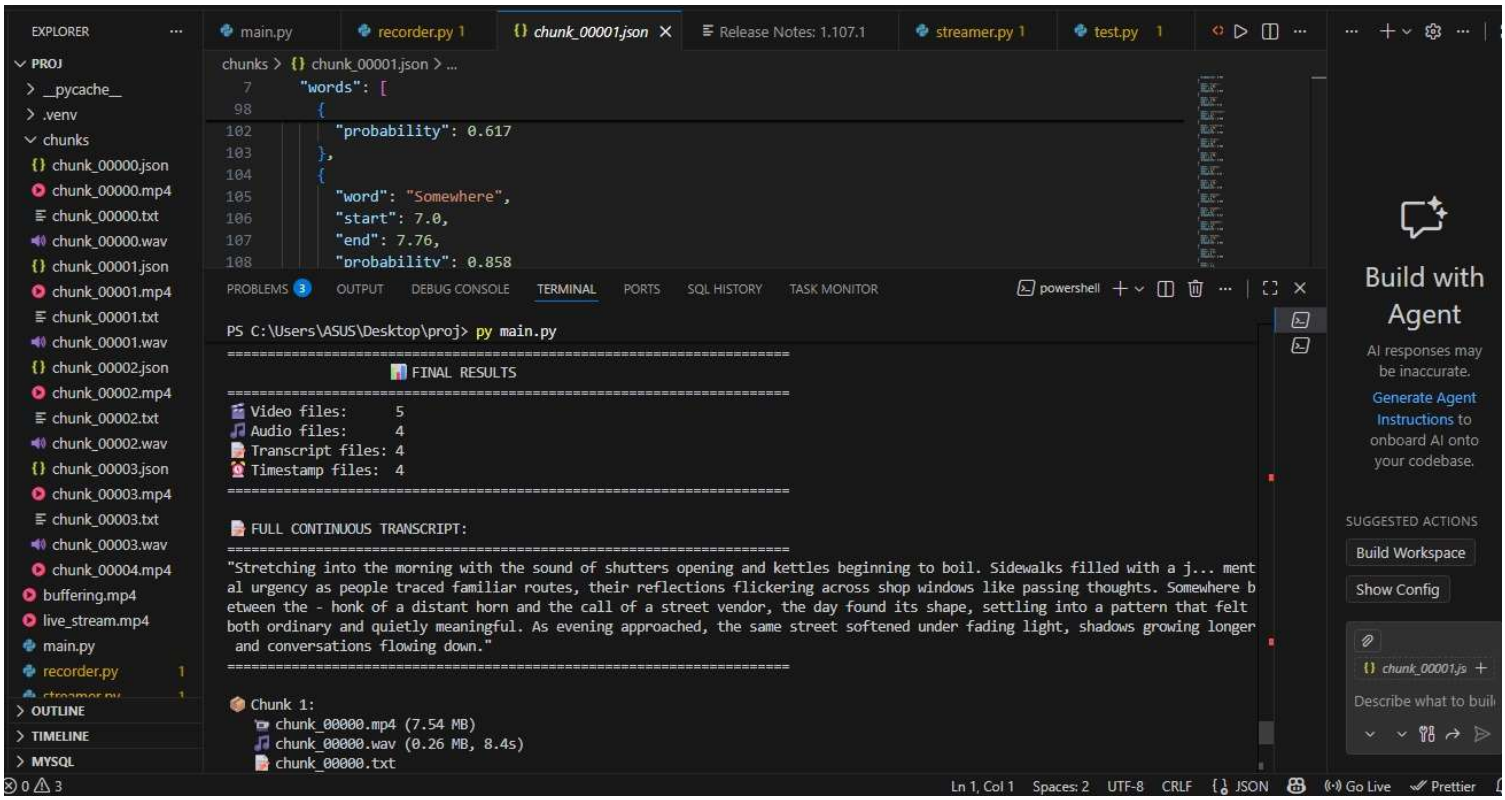
•Contains 311K labeled text samples.
•Text labeled as Offensive (1) or Non-Offensive (0).
•Data includes social media text with abusive language examples.

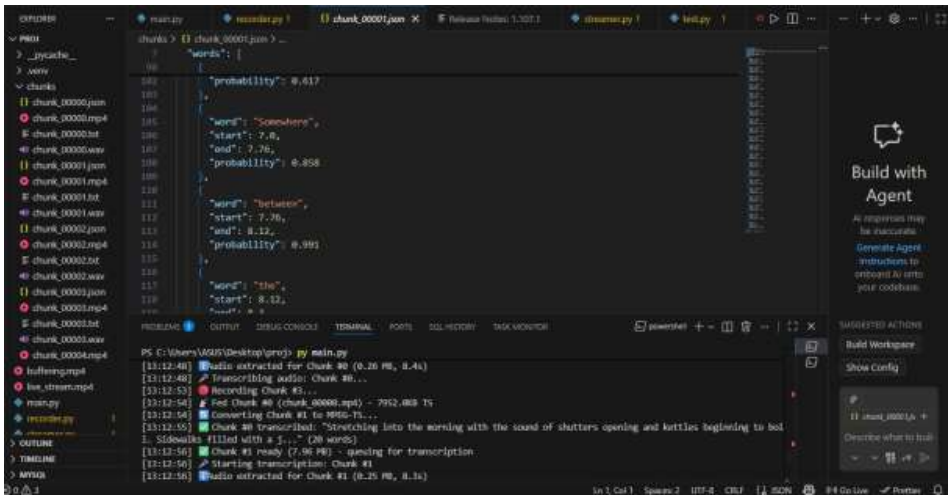
- Used to train context-aware DistilBERT model for live streaming moderation.

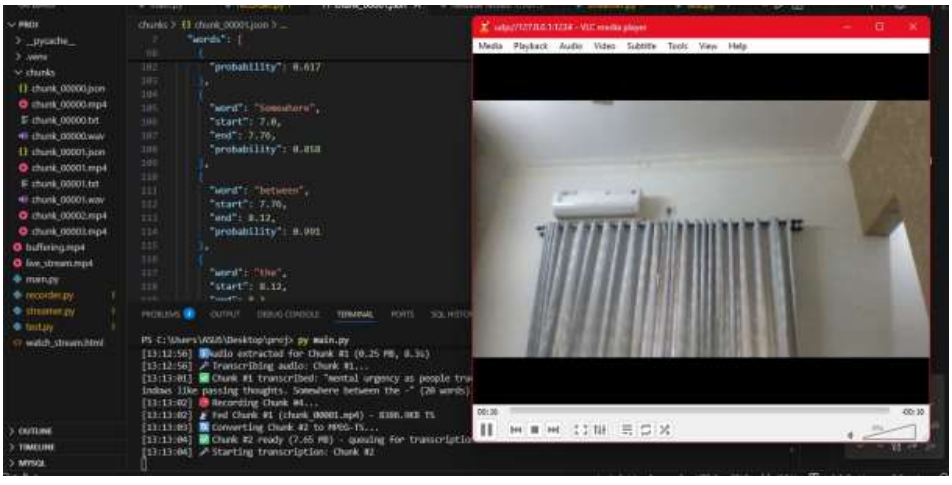
Automated Hate Speech Detection and the Problem of Offensive Language
Available on: GitHub / Kaggle

- Total Samples: 24,783 tweets
- Type: Social media text data
- Purpose: Detect hate speech and offensive language using NLP models

RESULTS


PS C:\Users\ASUS\Desktop\proj> py main.py
FINAL RESULTS
Video files: 5
Audio files: 4
Transcript files: 4
Timestamp files: 4
FULL CONTINUOUS TRANSCRIPT:
"Stretching into the morning with the sound of shutters opening and kettles beginning to boil. Sidewalks filled with a j... ment al urgency as people traced familiar routes, their reflections flickering across shop windows like passing thoughts. Somewhere b etween the - honk of a distant horn and the call of a street vendor, the day found its shape, settling into a pattern that felt both ordinary and quietly meaningful. As evening approached, the same street softened under fading light, shadows growing longer and conversations flowing down."
Chunk 1:
chunk_00000.mp4 (7.54 MB)
chunk_00000.wav (0.26 MB, 8.4s)
chunk_00000.txt
Build with Agent





```
            max_length=128,
            return_tensors="pt"
        ).to(device)

        with torch.no_grad():
            logits = model(**enc).logits

        return LABEL_MAP[logits.argmax(dim=-1).item()]
```

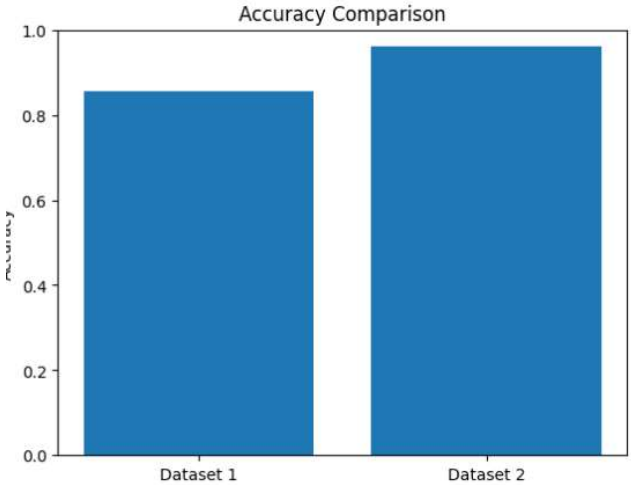
```
print(predict_sentence("you  are a bitch"))
print(predict_sentence("have a nice day"))
```

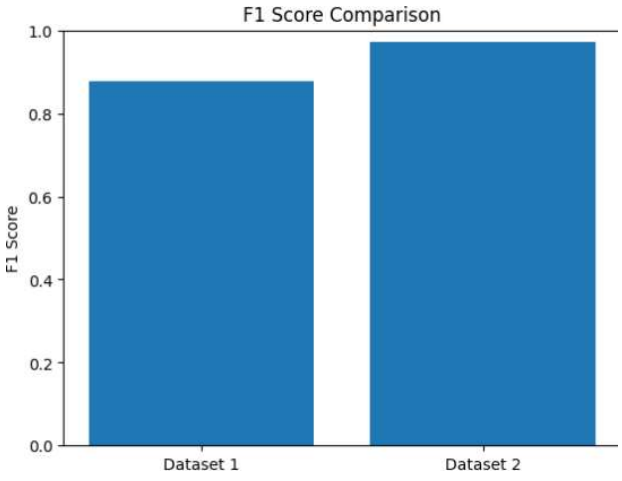
```
offensive
clean
```

```
        # Offensive trigger condition
        if label_with in ["hate", "offensive"] and label_without == "clean":
            offensive.append(words_with_timestamps[i])

    return offensive
```

```
words_with_timestamps = [
    {'word': 'you', 'start': 1.2, 'end': 1.35},
    {'word': 'are', 'start': 1.36, 'end': 1.45},
    {'word': 'a', 'start': 1.46, 'end': 1.52},
    {'word': 'idiot', 'start': 1.53, 'end': 1.8}
]

print(detect_offensive_words(words_with_timestamps))
```

```
[{'word': 'idiot', 'start': 1.53, 'end': 1.8}]
```



Assumptions

•The input audio is clear and speech is audible without excessive background noise.

•Profane words exist within the trained dataset or follow similar patterns.

•The system has access to a compatible GPU for real-time processing.

•Audio and video streams remain synchronized during recording and segmentation.

•Stable system resources (CPU, RAM, storage) are available for continuous operation.

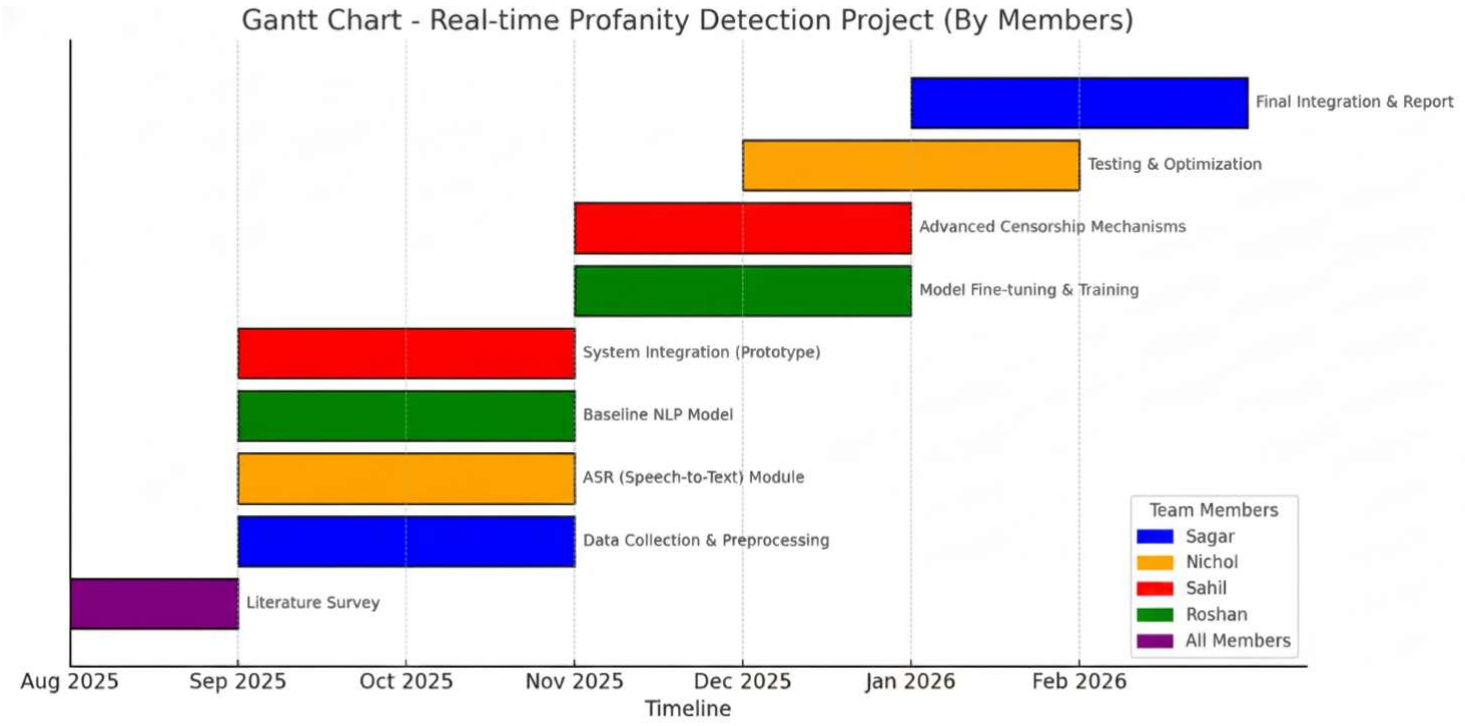
Role Of Each Individuals

- **Nichol:** Real-time Video Chunking- Splits a live video stream into small, manageable segments so each part can be processed instantly without waiting for the full video.
- **Roshan:** Training and fine-tuning Bert Model.
- **Sagar:** Audio-to-Text Transcription- Converts spoken audio in each chunk into text in real time, enabling language analysis like offensive word detection.
- **Sahil:** Real-time Streaming- Continuously delivers the processed video back to viewers with minimal delay, ensuring smooth live playback.

SOFTWARE REQUIREMENTS

- Python 3.10
- FFmpeg (video capture and segmentation)
- PyTorch with CUDA support
- Base-Whisper (Speech-to-Text)
- Transformers (BERT model)
- VLC Media Player (UDP stream monitoring)

ProjectSchedule



Risks And Challenges

- Real-time processing latency due to speech transcription and model inference
- High computational and GPU requirements for deep learning models
- Risk of false positives and false negatives in profanity detection
- Limited and evolving profanity datasets (new slang and context-based abuse)
- Audio quality issues such as background noise affecting detection accuracy

Conclusion

•This project implements a real-time AI-based system to detect and censor offensive language in live video streams.

•It combines speech-to-text transcription with context-aware Transformer-based NLP to accurately identify profane content.

•The system automatically applies instant censorship without interrupting the flow of audio and video.

•The final goal is to ensure safe, compliant, and audience-friendly content delivery for live streaming and digital media platforms.

References

[1] A. S. Ba Wazir, H. A. Karim et al., “Design and implementation of fast spoken foul language recognition with end-to-end dnns,” Sensors, vol. 21, no. 3, p. 710, 2021.

[2] Z. Zhao, Y. Wang, and Y. Wang, “Multi-level fusion of wav2vec 2.0 and bert for multimodal emotion recognition,” arXiv preprint arXiv:2207.04697, 2022.

[3] Y. Shang and T. Fu, “Multimodal fusion: A study on speech-text emotion recognition with deep learning,” Intelligent Systems with Applications, vol. 24, p. 200436, 2024.

[4] T.-X. Sun, X.-Y. Liu, X.-P. Qiu, and X.-J. Huang, “Paradigm shift in natural language processing,” Machine Intelligence Research, vol. 19, no. 3, pp. 169–183, 2022

[5] K. P. Gowda et al., “Transformers in sentiment analysis: A paradigm shift in deep learning research,” Journal of Information Systems Engineering and Management, 2024

[6] J. du Toit and M. Dunaiski, “Prompt tuning discriminative language models for hierarchical text classification,” Natural Language Processing, 2024.

[7] A. Loubser, P. De Villiers, and A. De Freitas, “End-to-end automated speech recognition using a character-based small-scale transformer,” Expert Systems with Applications, vol. 252, p. 124119, 2024.

[8] L. Ben Letaifa and J.-L. Rouas, “Variable scale pruning for transformer model compression in end-to-end speech recognition,” Algorithms, vol. 16, no. 9, p. 398, 2023.

[9] T. Guo, N. Yolwas, and W. Slamu, “Efficient conformer for agglutinative language asr using low-rank approximation and balanced softmax,” Applied Sciences, vol. 13, no. 7, p. 4642, 2023.

[10] R. Qasim, W. H. Bangyal, M. A. Alqarni, and A. Ali-Almazroi, “A fine-tuned bert-based transfer learning approach for text classification,” Journal of Healthcare Engineering, vol. 2022, pp. 1–12, 2022.

Thank You

Appendix B: Vision, Mission, Programme Outcomes and Course Outcomes

Vision, Mission, Programme Outcomes and Course Outcomes

Institute Vision

To evolve into a premier technological institution, moulding eminent professionals with creative minds, innovative ideas and sound practical skill, and to shape a future where technology works for the enrichment of mankind.

Institute Mission

To impart state-of-the-art knowledge to individuals in various technological disciplines and to inculcate in them a high degree of social consciousness and human values, thereby enabling them to face the challenges of life with courage and conviction.

Department Vision

To become a centre of excellence in Computer Science and Engineering, moulding professionals catering to the research and professional needs of national and international organizations.

Department Mission

To inspire and nurture students, with up-to-date knowledge in Computer Science and Engineering, ethics, team spirit, leadership abilities, innovation and creativity to come out with solutions meeting societal needs.

Programme Outcomes (PO)

Engineering Graduates will be able to:

1. Engineering Knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems

2. Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of

mathematics, natural sciences and engineering sciences

3. Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

4. Conduct investigations of complex problems: Use research-based knowledge including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

5. Engineering Tool Usage:Create, select and apply appropriate techniques, resources and modern engineering and IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems.

6. The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment..

7. Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national and international laws.

8. Individual and Team work: Function effectively as an individual, and as a member or leader in teams, and in multidisciplinary settings.

9. Communication: Communicate effectively with the engineering community and with society at large. Be able to comprehend and write effective reports documentation. Make effective presentations, and give and receive clear instructions.

10. Project management and finance: Demonstrate knowledge and understanding of engineering and management principles and apply these to one's own work, as a

member and leader in a team. Manage projects in multidisciplinary environments.

11. Life-long learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. .

Programme Specific Outcomes (PSO)

A graduate of the Computer Science and Engineering Program will demonstrate:

PSO1: Computer Science Specific Skills
The ability to identify, analyze and design solutions for complex engineering problems in multidisciplinary areas by understanding the core principles and concepts of computer science and thereby engage in national grand challenges.

PSO2: Programming and Software Development Skills
The ability to acquire programming efficiency by designing algorithms and applying standard practices in software project development to deliver quality software products meeting the demands of the industry.

PSO3: Professional Skills
The ability to apply the fundamentals of computer science in competitive research and to develop innovative products to meet the societal needs thereby evolving as an eminent researcher and entrepreneur.

Course Outcomes (CO)

Course Outcome 1: Model and solve real world problems by applying knowledge across domains (Cognitive knowledge level: Apply).

Course Outcome 2: Develop products, processes or technologies for sustainable and socially relevant applications (Cognitive knowledge level: Apply).

Course Outcome 3: Function effectively as an individual and as a leader in diverse

teams and to comprehend and execute designated tasks (Cognitive knowledge level: Apply).

Course Outcome 4: Plan and execute tasks utilizing available resources within timelines, following ethical and professional norms (Cognitive knowledge level: Apply).

Course Outcome 5: Identify technology/research gaps and propose innovative/creative solutions (Cognitive knowledge level: Analyze).

Course Outcome 6: Organize and communicate technical and scientific findings effectively in written and oral forms (Cognitive knowledge level: Apply).

Appendix C: CO-PO-PSO Mapping

CO - PO Mapping

CO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11
1	2	2	2	1		2	1	1	1	1	1
2	2	2	2		2	3	2	1	1	1	1
3								3	2	2	1
4		1	2		3	2	3	2	2	3	1
5	2	3	3	1	2	2				1	3
6			2			2	2	2	3	1	1

CO - PSO Mapping

CO	PSO 1	PSO 2	PSO 3
1	2	2	2
2	2	2	
3			3
4		2	2
5	3		
6			3

Justification

Mapping	**Justification**
CO1 - PO1	The project applies core computer science and engineering principles such as reinforcement learning (PPO) and LSTM-based modeling to identify and solve real-world problems in UAV communication.
CO1 - PO2	The study analyzes UAV handover challenges and designs a data-driven model integrating AI to ensure stable connectivity, demonstrating strong analytical and problem-solving ability.
CO1 - PO3	The project content involves interdisciplinary knowledge combining wireless networks, AI, and embedded systems, ensuring comprehensive problem understanding.
CO1 - PO4	Mathematical and algorithmic foundations are applied to develop and optimize the PPO agent and reward functions.
CO1 - PO7	The design supports sustainable drone operations by minimizing unnecessary handovers, reducing latency, and improving energy efficiency.
CO1 - PO12	Research and literature from IEEE and Expert Systems with Applications are analyzed to understand existing mobility management methods.
CO2 - PO1	PPO and LSTM-based algorithms are developed and optimized to enhance UAV handover prediction accuracy and real-time decision-making.
CO2 - PO2	The project translates research concepts into a system capable of sustainable and socially relevant applications such as disaster management and surveillance.
CO2 - PO3	The proposed multi-layer UAV-edge-cloud framework supports collaboration and coordination, enabling teamwork and leadership in system design.
CO2 - PO4	Project development included task planning within functional and non-functional requirements while following technical and ethical guidelines.
CO2 - PO7	The architecture emphasizes sustainable and secure data communication across UAV networks.

CO3 - PO5	The design involves team-based modeling, communication, and collaborative work using simulation tools and documentation.
CO3 - PO8	Tasks and module development are divided efficiently among team members, ensuring effective teamwork and communication.
CO4 - PO1	Timelines were managed for data collection, model development, and validation using structured documentation and presentations.
CO4 - PO10	Effective communication was achieved through well-prepared design and sequence diagrams demonstrating technical clarity.
CO4 - PO12	Continuous learning was maintained by updating models (PPO and LSTM) and retraining based on new datasets, supporting lifelong learning.
CO5 - PO5	The design process identified research gaps in UAV handover and proposed innovative reinforcement learning-based solutions.
CO5 - PO12	The study promoted creative problem solving through AI-powered mobility prediction, enhancing lifelong research capabilities.
CO6 - PO10	The technical findings, datasets, and performance improvements were communicated effectively through the final design documentation and presentation.
CO6 - PO11	Documentation, diagrams, and system design elements were organized professionally to communicate results clearly and concisely.

Mapping	**Justification**
CO1 - PSO1	Students identified and analyzed complex UAV handover issues and designed AI-driven solutions addressing national challenges in aerial communication.
CO1 - PSO2	The project applied algorithmic and programming skills (PPO, LSTM) to build a system capable of real-time decision-making in UAV networks.

CO1 - PSO3	The students conducted research integrating AI, networking, and embedded system concepts to design innovative, research-oriented solutions.
CO2 - PSO1	The UAV-edge-cloud architecture demonstrates engineering problem-solving and multidisciplinary integration.
CO2 - PSO2	Students implemented algorithmic optimization, enhancing software development and deployment efficiency.
CO2 - PSO3	The research design encourages application of computer science fundamentals for creating real-world impactful solutions.
CO3 - PSO1	The project promotes analysis and design thinking while preparing and presenting design solutions collaboratively.
CO3 - PSO2	Students demonstrated algorithmic proficiency in designing PPO-based control for UAV handover.
CO3 - PSO3	The design presentation reflects effective communication of complex research outcomes, nurturing research and entrepreneurship skills.
CO4 - PSO1	The layered design (UAV, Edge, Cloud) illustrates the ability to handle multidisciplinary challenges in system integration.
CO4 - PSO3	The system showcases professional and research skills by connecting AI models with sustainable UAV operations.
CO5 - PSO1	Students formulated innovative AI-based methods (PPO + LSTM) to solve real-world handover and energy issues.
CO5 - PSO3	Encourages innovation and research contributions toward smart 6G and UAV mobility management.
CO6 - PSO1	Students organized and communicated technical findings clearly using data-driven evidence.
CO6 - PSO2	Applied coding and simulation tools to develop and analyze AI models effectively.
CO6 - PSO3	Communicated complex technical findings professionally in written and oral formats.